Attack Tree Distance: A Practical Examination of Tree Similarity Measurement within Cybersecurity

NATHAN SCHIELE, Leiden Institute of Advanced Computer Science, Leiden University, Leiden, Netherlands
OLGA GADYATSKAYA, Leiden Institute of Advanced Computer Science, Leiden University, Leiden, Netherlands

Attack trees are a popular threat modeling method. In practice, there is often a need to compare attack tree models produced by human experts, based on both the structure of the tree and the meaning of the node labels. In this work, we investigate the problem of comparing attack trees and measuring their similarity. We define five different measures for measuring the distance between two attack trees: Label Distance (LD), Tree Edit Distance (TED), Radical Distance (RD), Multiset Distance (MSD) and Weighted Sum Distance (WSD). We further propose a repeatable method of both theoretical and experimental attack tree distance measures validation. Our theoretical validation consists of a series of basic transformations to evaluate the behavior of distance measures with respect to specific types of transformations that may appear between two attack trees. To experimentally validate our distance measures, we designed and executed a human study (n = 39) to collect a dataset of attack trees to be used for evaluation and comparison of the measures.

From our theoretical and experimental results, we find that applying semantic similarity as a means of comparing node labels is a valid approach. Further, we find four of the five attack tree distance measures are valid approaches in certain, varying circumstances. Our results suggest that these methods can already be used to identify similar real-world attack trees. Overall, this work lays the groundwork for improved threat model analysis, validation of AI-generated attack trees, and future research into threat similarity measurement in cybersecurity.

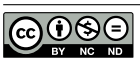
CCS Concepts: • **Security and privacy** → **Formal security models**; **Usability in security and privacy**; Security requirements.

Additional Key Words and Phrases: Attack Trees, Threat Models, Tree Distance, Semantic Similarity

1 Introduction

Threat modeling seeks to organize threat information that aids in comprehension and analysis of cyber attacks [6, 129]. *Attack trees* (ATs) are a popular method for threat modeling in software engineering [133, 140], security requirements elicitation [86, 117], and security risk assessment [52, 110]. ATs represent attack scenarios as labeled rooted trees, where the root label captures the high-level attacker's goal, and the labels of all other nodes represent particular sub-goals or steps to achieve them. There is a rich literature on attack tree models designed by experts to better understand threats to a particular system, e.g., ATMs [33], smart cars [70, 120], cloud systems [30, 148], smart grids [9, 90], medical implants [134] or SCADA systems [141]. Furthermore, in the past few years, there has been a substantial interest in the automated synthesis of attack trees [71] using formal system models [8, 35, 53, 71, 109, 115, 137, 145, 150], libraries of attack patterns [14, 15, 41, 58], or even machine learning methods [68, 138] and, most recently, large language models (LLMs) [2, 36, 84].

Authors' Contact Information: Nathan Schiele, Leiden Institute of Advanced Computer Science, Leiden University, Leiden, ZH, Netherlands; e-mail: n.d.schiele@liacs.leidenuniv.nl; Olga Gadyatskaya, Leiden Institute of Advanced Computer Science, Leiden University, Leiden, ZH, Netherlands; e-mail: o.gadyatskaya@liacs.leidenuniv.nl.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.
© 2026 Copyright held by the owner/author(s).
ACM 2471-2574/2026/9-ART
https://doi.org/10.1145/3850236

Just Accepted

Given this proliferation of attack tree models produced for different scenarios, especially when they are synthesized automatically, novel research problems emerge. *How can we establish whether two attack trees produced by different experts or with different synthesis methods describe similar attack scenarios?* For example, consider the recent work by Nicoletti et al. [103] that proposes a method to represent MITRE attack campaigns as attack trees and compares these campaigns based on computed quantitative attributes; it would be very useful to quantify the similarity of different campaign models to identify threat actors that have related modi operandi. In his foundational paper, Schneier advises creating a library of reusable attack trees that could capture relevant attack scenarios, thus diminishing the need to have security experts around [128]. Researchers have started to crowd-source attack trees for benchmarking [29]. In such a setting, when there is a collected library of attack trees, an automated way to identify already present attack scenarios similar to the newly submitted one will help categorize and curate new additions. A related novel research problem was investigated in [36]: *how to establish the quality of an LLM-synthesized attack tree by comparing it to a tree designed by a human expert?* However, currently, there is no standard method for comparing attack trees or measuring their similarity. Existing attack tree semantics can partially answer this question as semantics provide a way to define the meaning of an attack tree model [89]. However, an AT semantics-based approach can only conclude whether two attack trees are equivalent: the established attack tree semantics do not allow estimating similarity between these trees or comparing tree models with similar yet unequal labels that might appear identical to a human [57, 72, 89]. While other tree (and tree-like) semantic representations allow for a similarity estimate as opposed to an equivalence estimate [24], such semantic representations have not been shown to apply to attack trees. Consider, for example, two attack goals *rob a bank* and *steal money from a bank*. To a human expert, these two attack goals might appear quite similar. A strict semantics-based approach, e.g., *à la* Mauw and Oostdijk [89], will consider these to be two different attacks. We thus aim to propose new methods that would automate the assessment of AT similarity.

Research question and contributions. Our fundamental research question (RQ) is: *How do we measure the distance between two human-created attack trees?* Examining this question leads to three sub-research questions:

(1) **SRQ1**: How to compare human-created sets of attack tree node labels?
(2) **SRQ2**: How to compare the structure of attack trees?
(3) **SRQ3**: How to validate a distance measure for attack trees?

To answer these questions, we first propose a method for comparing the labels of attack trees based on semantic similarity using SBERT [27, 119]. This is then refined into our first suggested distance measure called *label distance*, which estimates how close the whole sets of labels of two trees are. **Our first contribution is a repeatable method for comparing two lists of labels based on semantic similarity**.

We then apply the proposed *label distance* and develop an approach for comparing the structure of ATs. We propose three additional methods for calculating the distance between two attack trees that incorporate structural elements of the tree: (1) *tree edit distance* [159]; (2) *radical distance* inspired by [125] and the refinement semantics discussed in [35]; and (3) *multiset distance* that stems from the multiset attack tree semantics proposed in [89]. We further propose the fifth method, which is a *weighted combination of these three distance measures*. **Our second contribution is these four methods for comparing the structure of two attack trees**.

Following this, we provide a method for both theoretically and experimentally validating any attack tree distance measurement method to ensure that the distance measure reflects how best understanding of how two trees differ. Specifically, we establish a complete set of basic transformations that a tree could undergo, and assess our distance measures with respect to these transformations. We also collect a series of human-drawn ATs and compare them using our distance measures. Finally, we experiment with multi-dimensional scaling [142] as another approach to demonstrate the validity of the proposed methods. **Our third contribution is a repeatable method for validating any attack tree distance measure**.

Just Accepted

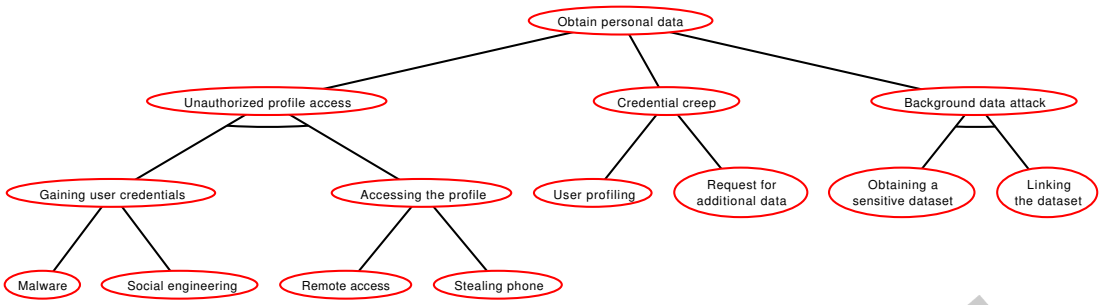


Fig. 1. An attack tree adapted from Naik et al. [101] that is used in the study described in Section 6.1.1.

We believe that in the future, when we can expect attack models to be synthesized using dedicated generative AI methods [31, 43, 152, 156, 160] , similarity measurement for ATs will become an even more important problem. Our work paves the way to solve this problem and to develop other, even more robust attack tree similarity measurement methods.

The structure of this paper. We first introduce some background concepts about attack trees (Section 2) and discuss the related literature on tree distance metrics (Section 3). We then define an idea of *semantic similarity* for node labels that allows matching realistic, "*noisy*" labels and relaxes the strict label equality requirement (Section 4). We conceptualize and define 4 distance measures for attack trees (Section 5). Subsequently, we propose our validation methodology (Section 6), and we report the results of evaluating these measures following our validation method (Section 7). We then analyze which of the metric axioms are satisfied by the proposed distance measures (Section 8), discuss our key findings in Section 9, and conclude with Section 10.

2 Background

We define attack trees (ATs), as originally described by Bruce Schneier [128], to be a rooted acyclic structure with the following recursive definition from [35].

An attack tree t is defined as:

$$t = b|b\Delta(t_1, ..., t_i)$$

Where $b \in \mathbb{B}$ is an action or goal in service of an attack, considering $\mathbb{B}$ to be the set of all possible attack components. For our purposes, b is considered to be the label of a node. Δ is the refinement operator of a node, defining the relationship between child nodes. In this work, Δ is either AND or OR.

Traditionally, the root node of an attack tree represents the main attacker's goal, and the lower nodes represent *refinements* of the main goal into various attack steps and options available [89]. The AND refinement means that all child nodes must be achieved in order to succeed at the parent node level; while the OR refinement means that the attacker has options, and achieving any of the child nodes is enough to succeed at the parent node level. The lowest nodes in the tree that are not refined are called leaf nodes.

An example of an attack tree describing how personal data can be obtained is available in Figure 1. The root node is *Obtain personal data*, which is refined using an OR operator into three sub-goals: *Unauthorized profile access*, *Cred. creep*, and *Background data attack*. The first sub-goal of *Unauthorized profile access* is further refined using an AND operator into two options: *Gaining user credentials* and *Access profile*. The first of these is further refined using an OR operator into two steps: *Malware* and *Social engineering*. These nodes are not further refined and are thus leaf nodes. Each node is labeled with a short textual description of the attack step or goal the node represents. The specificity of the labels can vary, and will fundamentally depend on the application context and the user designing the tree.

Just Accepted

Semantics attempt to represent the meaning of an attack tree and enable equality testing: if two trees can be represented by the same Boolean formula or the same set of multisets, then they are equal in the considered semantics. If two trees are semantically equivalent, then all quantitative analyses with attributes compatible with the semantics will result in equal values [89]. Trivially, this semantics-based reasoning can be extended from equality to testing for inclusion (being a subset). However, it is not evident how to extend the semantics-based approach to measuring similarity or distance between two attack trees.

Indeed, to consider the similarity of attack trees, we need to take into account two key aspects of this model: *node labels* (including labels of intermediate nodes that are absent from, for example, the multiset or propositional semantics [89]) and the *refinement structure* that defines what attacks are subcomponents of which other steps. Moreover, as we mentioned, in real life, node labels are somewhat *noisy*: experts (or LLMs [36]) can propose slightly different textual labels to define the same underlying attack idea. For example, when brainstorming about the same attack scenario captured in Figure 1, one expert can use the label *Obtain personal data* for the root node, another person – *Obtain private user data*, and a third expert – *Get private information*. Thus, to be able to compare real attack trees (which we also call *"unfiltered"* in this work), we need to develop means to account for this variation ("noise") in the natural language.

3 Related Work

3.1 Natural Language Processing (NLP) and Semantic Inference

Natural language processing is the area of computer science focused on machine processing of human-generated language [21]. There are multiple subfields within NLP focusing on different avenues of machine analysis of human language [16, 46]. For example, syntactic analysis focuses on leveraging the structure of language to understand meaning [135]. Semantic analysis, on the other hand, focuses on the meaning of language [62].

There are multiple methods to process human language semantically. A common method involves converting individual words to semantic meaning vectors using a pre-trained Recurrent Neural Network (RNN) using Word2Vec developed by [97]. Palmer et al. extended syntactic analysis by adding semantic role labels (information concerning predicates) [108]; a work later expanded upon by Collobert et al. who created a neural network using, in part, semantic role labels to comprehend whole sentences [23]. Han et al. proposed a method for extracting semantic features of words and classifying them using a Long-Short-Term Memory (LSTM) network [42], while Turian et al. devised a semi-supervised approach to achieve the same goal of semantic representation of words [144]. Kalchbrenner et al. created a Convolutional Neural Network (CNN) to model the semantic meaning of sentences [61], which uses tree-structured sentence representations to make semantic comparisons. However, there is no clearly established method to definitively compare the meaning of sentences [21].

Chandrasekaran and Mago performed a survey of semantic similarity techniques in literature, exploring the different methods used to calculate the semantic similarity between two text elements [18]. Our research most closely aligns with their proposed classification of Explicit Semantic Analysis (ESA) type of Corpus-based semantic analysis, akin to the work by Gabrilovich and Markovitch [34]. This type of semantic similarity creates a vector-based representation of text (in our case, node labels) and then compares those vectors with cosine similarity. However, many other methods of semantic comparison could be applied for measuring the similarity of two text elements. For example, Information Content (IC) compares taxonomies of data [59, 121] and significant developments have been made toward improving the IC semantic similarity measure [25, 26, 93, 116, 151].

One of the most critical developments in semantic word representation is the development of Bidirectional Encoder Representations from Transformers (BERT) from Devlin et al. [27]. BERT is a transformer-based model that is pre-trained on a large corpus of text. It is capable of understanding the context of a word in a sentence and can be fine-tuned for a variety of NLP tasks. Liu et al. replicated and improved the results of BERT in RoBERTa [82].

Just Accepted

Natural Language Sentence Mapping (NLSM) is a specific research area within NLP that focuses on comparing sentences with each other and predicting matching sentences [85, 149]. NLSM matches the challenge we tackle in comparing the labels of two trees, discussed in detail in Section 4. Hu et al. uses a CNN to compare sentences [48], while Wang et al. proposes a multi-perspective matching model to do the same [149]. Several works have expanded BERT specifically for the task of sentence matching. Zhou et al. extends BERT to introduce relation awareness [161], while Liu et al. introduces syntax and semantic awareness to BERT [80]. Although many of these approaches would be easily applicable for our goals to establish semantic similarity of AT labels (Section 4), given the lack of a clearly established applicable method for NLSM [81], we seek an approach that is both easily reproduced and iterated upon (for example, by using either a pre-trained model or a completely different, task-specific model).

We use the Sentence-Bidirectional Encoder Representations from Transformers (SBERT) from Reimers and Gurevych [119]. This is a state-of-the-art sentence embedding model built upon the well-established BERT model. We use several pre-trained SBERT models for our semantic analysis. We take inspiration from Tracz et al. who used BERT in a product matching solution [143]. Their contribution was the novel product matching solution, not the application of BERT. Similarly, our contribution is the distance measures, not the application of SBERT in the distance measures.

3.2 Tree Edit Distance

Distance between data structures is not a new concept. Many works have explored the idea of "distance" between strings. In string edit distance, the difference in strings is given by a min-cost path taken by either adding a character, removing a character, or replacing a character. By finding the minimum cost needed to transform one string into another, a "distance" value can be given. The cost of transformation increases with the difference between two strings [88, 155].

Tree edit distance (TED) is seen as an extension of the string edit distance. Tai first tackled the problem, suggesting an edit distance metric for two directed acyclic graphs (DAGs) [139]. Zhang and Shasha have written the seminal work on tree edit distance [158]. We describe this distance and our application of it in more detail in Section 5.1.

Most of the research and development on tree edit distance focuses on optimizing computations. As shown by Zhang et al., the tree edit distance problem for unordered trees is an *NP*-Complete problem [159]. As such, the development of novel optimal calculation strategies is necessary to enable the comparison of larger tree structures. Yoshino et al. developed a dynamic programming A* algorithm for computing unordered tree edit distance (UTED), which offers significant performance gains over exhaustive search [154]. Their method uses an A* algorithm to construct a search tree of mappings. The distance is then calculated from these mappings. Unfortunately, these mappings rely on an absolute equivalence between nodes of two trees to establish a mapping, which cannot be established a priori for "unfiltered" (real-world) attack trees, and thus cannot be applied in our case. Other works based on the A* algorithm methodology, such as the optimizations offered by Paassen [107] for computing UTED, will have the same drawback. However, the intuition to first find a potential optimal mapping and then find the distance based on the mapping is a useful methodology that we can apply to our work.

McVicar et al. have developed a method of calculating UTED in SuMoTED by focusing on allowing nodes to move up or down a tree and organizing edits around a consensus tree [92]. This methodology allows them to achieve polynomial-time calculation of UTED. However, the application of this methodology to attack trees presents difficulties. Namely, we have the same issue as the A* algorithm: the requirement of exact equivalence between node labels.

Pawlik and Augusten proposed RTED, a more optimal method of calculating ordered TED (OTED) [111]. Their methodology is faster than Zhang and Shasha [158], but specifically for larger trees (500+ nodes). Attack trees,

Just Accepted

by contrast, do not grow this large as they become cognitively too demanding as threat models [6]. Thus, as such, RTED does not offer a significant advantage for attack trees over Zhang and Shasha [158].

3.3 Tree Kernels

A common way to calculate tree distances in machine learning is the use of tree kernels. Tree kernels compare tree structures by computing some positive-definite function over a subset of features of the trees (for example, the dot product). There is no commonly accepted method for selecting features or selecting the positive-definite function [132]. A distance measure must already be clearly defined in order to build a tree kernel [131]. Moschitti proposed a mechanism for applying tree kernels to NLP by representing sentences with syntactic parse trees and comparing those trees [98]; Collins and Duffy proposed a similar method for comparing parse trees [22]. Tree kernels have been used for semantic analysis, such as by Aseervatham et al. [7].

Building on Collins and Duffy, Kashima and Koyamada proposed a method of comparing trees by comparing the substructures of the trees [64]. This method is based on the idea that the similarity of two trees is the sum of the similarities of their substructures. Shin built upon the work of Kashima and Koyamada by comparing trees by matching substructures, a matching methodology similar to what we employ [131]. These methods informed our development of the radical distance, described in Section 5.2. In almost all cases, tree kernels either only compare structural information or are used to represent non-tree structured information (such as sentence structure). Tree kernels have been used for both NLP and structural tree comparison, but have not been used for both simultaneously; our work is essentially this. To our knowledge, no previous work has attempted to apply tree kernels to unfiltered tree-structured data. It may be possible to develop a tree kernel that can both compare the structure of the tree and the meaning of the node labels (*e.g.* a nested tree kernel), but this is outside the scope of this work.

3.4 Tree Similarity Measures in Other Fields

Tree distance applications appear in many fields. As already mentioned, tree edit distance is an important component of NLP and is used in many different NLP works to achieve machine comprehension, with Palmer et al. comparing tree representations of semantic roles [108], Kalchbrenner et al. comparing syntax trees [61], and Liu et al. comparing tree representation of sentences [81].

In other fields, for example, tree (edit) distance has been used by Otu and Sayood to compare phylogenetic trees in biology [105], by Chen and Zhang to compare RNA secondary structures [19], by Xu et al. to detect fake web sites by comparing tree representations of the DOM structure [153], by McNamee to compare patent classifications (stored in hierarchical tree structures) [91]. A work from Lakkarju et al. especially informs our work, as it shows a mechanism to compare XML files based on tree distance (creating tree representations of XML tags, but not the content of the tags) [75]. We believe that our work could fill a similar gap, or offer a pathway toward filling this gap, in the field of threat modeling.

3.5 Applications of Distance Measures in Cybersecurity

Distance measures are an often-discussed topic within cybersecurity; however, distances between threat models are not a well-explored area. The most directly related work to ours is by Lo et al. which used a variety of distance measures to determine insider threats within a CERT dataset [83]. One of their distance measures included the Levenshtein distance that we explore in Section 4.2. Related to this is a work by Rashid et al., who used Hidden Markov Models to accomplish the same [118]. A common discussion of distance comes from discussions of security metrics, as described by Andrew Jaquith: if security is quantified into metrics, these metrics can be compared with distance measures [55]. This was more closely examined by Krautsevich et al., proposing several mathematical formalisms for determining security hierarchies [74]. Similar work to Krautsevich et al. has been

Just Accepted

performed by Wagner et al., who focus on using distance measures for privacy comparisons [146]. Measuring the difference between Common Vulnerabilities and Exposures (CVEs) and Common Weakness Enumeration (CWEs), and mapping CVEs and CWEs to malicious behavior is another active area of research in cybersecurity [41, 44, 68]. These vulnerability metrics were applied by Wang et al. to another popular graphical security model, attack graphs (AGs), which are further discussed in the next section, [147]. To our knowledge, we are the first to examine distance as it is applied to attack trees.

3.6 Other Threat Models

While attack trees are a prominent threat modeling technique [133], other threat modeling techniques are also widely used and studied [126]. Most closely related to attack trees are fault trees, from which attack trees were designed [128]. Fault trees represent faults using logical structures, similar to how attack trees represent attacks [76]. STRIDE and LINDDUN are popular text-based threat modeling strategies for evaluating system threats [12, 50, 66]. Description similarity-based clustering of privacy-specific threats elicited using the LINDDUN threat modeling method were investigated by Bellemans et al. [10]. CORAS is another graphical threat modeling technique that represents attack scenarios by classifying different elements of the attack and visually showing the relationships between these elements [136].

The threat modeling technique that is most closely related to ATs is perhaps attack graphs (AGs), which are a state-based representation of attacks [130], where nodes represent system states and edges represent actions that transition the system from one state to another [114]. The research surrounding AGs is particularly robust, ranging from works focusing on optimizing the visual representation of AGs, as, e.g., Homer et al. [47], and those developing more efficient applications of AGs as a tool for analyzing threat information, as, e.g., Munoz-Gonzalez et al. [99]. Ou et al. created MulVAL, a common tool for creating AGs [106]. Jha et al. has formally analyzed the mathematics of AGs [56], while Ammann et al. brought a new development in the introduction of a monotonicity requirement [5]; both examples of works seeking to better understand the foundations of AGs as a formalism. Katsikeas et al. has focused on the empirical evaluation of the efficacy of AGs [65], basing their work on prior meta-analysis by Johnson et al. [60]. Kaynar has described a taxonomy for the application and generation of AGs [67] , which is an extension of the topological analyses conducted by Jajodia et al. [54] and Noel et al. [104]. Further analysis of AGs includes frequency analysis, as demonstrated by Albanese et al. [4].

Bopche and Mehtre tackled a similar issue to that presented in this work in their comparison of attack graphs; however, their work focused on two graphs of the same system after time had passed, and was intended to measure temporal differences in systems [13]. As such, like with much of the previous work on tree similarity, they can make assumptions about the equivalence of nodes present in any two AGs; further, they do not attempt to compare two different systems. However, their use of graph edit distance (GED), an expansion of the tree edit distance (TED) we use, helps inform how such a comparison is possible. While this helps inform our use of TED, fundamentally, AGs are different from ATs in that AGs do not present attack information in a hierarchical manner [133], while our AT distance measures aim to capture some elements of this hierarchical structure. Remarkably, Zhang et al. shows why a distance metric could be of use, in their work on using LLMs to boost AG construction, where a distance metric would aid in the validation of LLM contributions [160]; we believe an attack tree distance measure would serve a similar purpose.

Finally, prior works by Pinchinat et al. [115], and Schiele and Gadyatskaya [125] have established a way to conduct transformations between AT and AG models, thus enabling an application, through transformation, of our distance measures on AGs. Pinchinat et al. defined the conversion of AGs to ATs with AtSyRa [115], while Schiele and Gadyatskaya defined the reverse transformation [125]. This means that developments for one threat model (in the case of this work, ATs) can be directly applied to AGs in the future.

Just Accepted

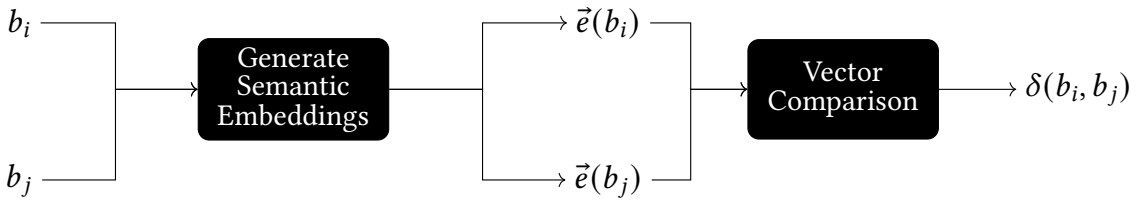


Fig. 2. Process of calculating the distance between two node labels.

Novelty of this work. To the best of our knowledge, there are no proposals in the literature on how to compare attack trees or to measure their similarity, or to compare tree-structured data with noisy, human-made labels. This is the gap we are aiming to close with this work.

4 Semantic Similarity

In this section, we address our first research question, **SRQ1**, on how to compare and measure distance between noisy attack tree labels and sets of such labels.

4.1 Label Sets

As there are no methods for comparing real-life tree-structured data, we develop our own approach. Following [11], which focuses on specific node labels to help analysis, and [45], which suggests a mechanism to decompose tree structures, we propose a robust and repeatable method for comparing unfiltered data that is organized in a tree structure. We propose to compare the sets of labels of the tree structure to form a mapping similar to the suggestion of Zhang and Shasha [158]. This mapping can be used in lieu of absolute equivalence. We create the label sets by extracting all node labels from an AT. As such, a comparison of the two label sets will amount to a comparison of the node labels without any structural information. Thus, if we were to have two trees with identical structures but different node labels, the comparison of these two label sets should be a sufficient proxy for the comparison of the trees from which those sets were generated.

4.2 Comparing Labels

Fundamental to comparing label sets is comparing the labels within those sets. We discussed much of the state-of-the-art of language processing in Section 3.1. We start from a more basic premise: directly comparing the labels as strings. A common method of comparing strings is to use string edit distance (Levenshtein distance) [77, 122]. Levenshtein distance will count the number of changes needed to be made to the strings to convert one string to another. This causes larger values for longer strings (as more edits are needed). However, this introduces a problem: longer strings will tend to have larger distances than shorter strings due to the increased number of comparisons.

The Levenshtein distance can be normalized by dividing the resulting distance value by the largest string length (a process previously discussed in [87, 155]), yielding a percentage measurement of the number of characters in a string that match. This allows for a direct comparison of two labels as strings, which yields a value between 0 and 1 regardless of the length of the strings. The following equation yields the normalized string edit distance, for two labels b_i and b_j:

$$\delta\left(b_i, b_j\right) = 1 - \frac{\text{Levenshtein}\left(b_i, b_j\right)}{\max\left(\left|b_i\right|, \left|b_j\right|\right)}$$

Just Accepted

However, Levenshtein distance does not consider the meaning of the labels, which is more fundamental to how we would like to compare the labels. We wish to develop measures that can reflect how similar two attack trees are in *meaning* rather than simple string distance. As such, we opt for comparing the semantic meaning of the labels using SBERT [119] to create a vector representation of the labels. We can then compare these vectors using cosine similarity, which is a measure of the angle between two vectors [78]. This gives us a value between 0 and 1, and is directly comparable to using the normalized Levenshtein similarity. The process of calculating the semantic similarity is shown in Figure 2. We further explore the negative effects of using Levenshtein distance in Section 7.1.

4.3 Comparing Label Sets

With an established mechanism to compare two labels, we move on to the problem of comparing sets of labels. We do this by taking the Jaccard distance between two sets of labels [73]. The Jaccard distance is a measure of similarity between two sets, and is calculated as the size of the intersection of the two sets divided by the size of the union of the two sets [38]. However, the Jaccard distance requires a binary determination of an equivalence between two labels. We thus first use the semantic similarity measure we defined above to determine the similarity of two labels, and then compare this similarity against a threshold ϵ. We discuss this threshold in the next section.

4.4 Equivalence Threshold

Equivalence is a binary measurement: labels are either equivalent or not. To ensure comparability of distance measures, all of the AT distance measures we conceptualize further in Section 5 require binary determinations of equivalence between pairs of labels. While it may be possible to establish distance measures that allow for more nuanced determinations of similarity (*i.e.* not binary), this is outside the scope of this work. We are interested in establishing a mechanism to compare distances between attack trees and validate those measures. We leave the refinement of attack tree distance measures to future work.

However, the semantic similarity we defined above is a continuous value between 0 and 1, which by its nature is not binary. To convert this continuous measure of semantic similarity to one of semantic equivalence, we employ a threshold value of ϵ. Two labels with any semantic similarity value above ϵ are considered equivalent, and labels with a semantic similarity value below ϵ are considered not equivalent.

Thus, we encounter the threshold problem: the problem of defining a threshold ϵ for the semantic similarity between two labels. This threshold is necessary to determine if two labels are similar enough to be considered the same. We further propose a way to determine a suitable range for the threshold based on empirical results with our attack tree datasets. Currently, the same threshold ϵ is applied to all pairs of labels. In the future, it might be interesting to explore whether more nuanced interpretations of semantic similarity (e.g., varying the threshold for different label types) enhance the attack tree similarity measurement approaches.

In Section 7.2, we discuss the process of finding the right ϵ for a given dataset. This process involves iteratively examining the dataset to establish what equivalences need to be assumed in order to create a meaning distance measure. From these equivalence assumptions, an optimal ϵ can be established. This process is similar to the process of establishing a threshold for a binary classification problem, where the threshold is set based on the desired false positive and false negative rates [123]. Our exact process is visually represented in Figure 7.

4.5 Label Distance

We now define the Label Distance (LD) – a distance measure between the sets of labels of two attack trees; LD is functionally a Jaccard distance on two sets of labels. We derive this measure from the mappings suggested in Tai [139] and Zhang and Shasha [158], with an algorithm influenced by the A* tree edit distance algorithm

Just Accepted

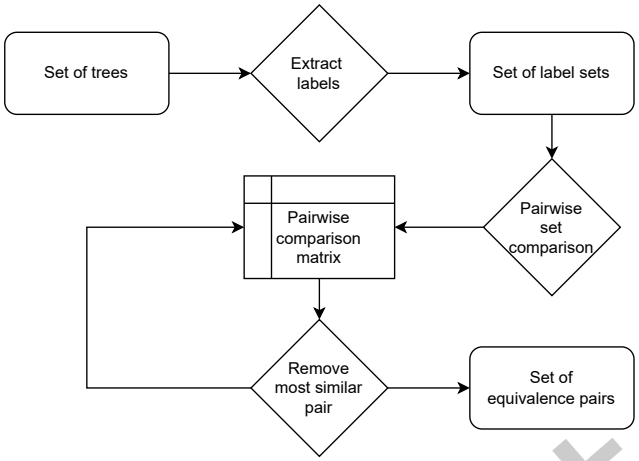


Fig. 3. Flowchart of the steps in label distance

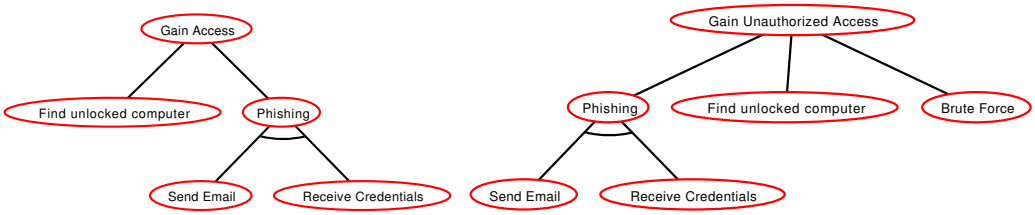
from Yoshino et al. [154]. A pre-trained SBERT model is used to calculate the distance between all label pairs, following the semantic similarity comparison introduced in Section 4 as shown in Figure 2.

We then create an $M \times N$ matrix D, which represents all the distances between all label pairs for two trees. The largest value at index i, j of this matrix is taken, and the corresponding row (i) and column (j) are removed from D, while recording the mapping between b_i and b_j. This is repeated until all nodes are included in a mapping set. If the largest value in D is below some value ϵ, we consider the labels to be different, and a direct mapping is not created. We add the nodes as individuals to the mapping (mapping to/from Λ, indicating that these labels would be removed or added). We then calculate the cost of each node that would be removed or added, considering these operations at the unit cost of 1. This process is shown in Figure 3. The total cost of the mapping is the distance between the two sets of labels. The normalized measure is calculated by taking this distance value and dividing it by the number of nodes in the largest tree.

This distance measure considers all the node labels within an attack tree. It uses a semantic comparison to examine the meanings of the nodes, which would allow for this distance measure to work on unfiltered trees. However, this measure does not consider the structure of the tree in any way and, therefore, does not represent how the nodes are organized. If we examine the attack tree in Figure 1, for an attack tree with all the same label names, but all nodes directly attached to the root (a single root node with 13 child nodes with identical names as in Figure 1), the resulting label distance would be zero. Thus, as such, label distance offers a way to quickly examine if the meanings of the nodes are similar, but does not offer a way to examine the structure of the tree.

Applying LD to the examples provided in Figure 4, we would first create two lists of labels of both Figures 4a and 4b. Then we could compare these lists according to their semantic similarity along the process described in Section 4. The labels that are identical will be matched first, then the labels will be matched according to their semantic similarity in decreasing order. This is repeated until only the node "brute force" from Figure 4b remains. Provided all other nodes match (given a sufficiently permissive value of ϵ), LD will result in a distance of 1 (normalized distance of 0.167).

Just Accepted



(a) A simple attack tree. (b) A slightly modified attack tree.

Fig. 4. Two similar attack trees with small differences to model the distance measures.

5 Distance Measures

We have established a mechanism for comparing node labels and using this mechanism to compare the label sets of two attack trees. However, these comparisons do not take into account the structure of the attack trees; we address including structural information in this section. In each of the following distance measures, we make use of the same semantic similarity measurement we use for the label distance to compare node labels. These results will answer our research question **SRQ2** about providing means of comparing the structure of attack trees, while also considering the label similarity. As a running example to demonstrate how these proposed distance measures work, we refer to the two similar attack trees shown in Figure 4.

5.1 Tree Edit Distance

Tree edit distance (TED) is a measure of the difference between two trees by defining an optimal edit path; that is, what changes are needed to turn one tree into another. The seminal work in this field is by Zhang and Shasha [158]. This was developed from, and is highly based on, the previously discussed string edit distance (Levenshtein distance) [139]. In their work, they describe a simple algorithm for calculating the distance between two trees [158]. This algorithm is based on the idea of a *forest* distance, which is the distance between two forests, or sets of trees. The result of this algorithm is a measure of the costs of edits needed to transform one tree into another, with the convention being that any edit requires a cost of 1. The possible edits are:

- matching, in which two nodes are defined to be equivalent, which is the only edit without cost,
- insertion, in which a node is added,
- deletion, in which a node is removed, and changing one node into another, in which a node has its label replaced.

For the edits, we need to know if two nodes are equivalent. This is done by comparing the semantic similarity of the two nodes; this is performed in the same manner as we did for the label distance. If the semantic similarity is above a given threshold, we consider the nodes to be equivalent; otherwise, we consider them to be different. As the TED value will provide us with a number of edits needed to transform one tree into another, we can normalize this value by dividing it by the number of nodes in the tree with the most nodes. This will give us a normalized value between 0 and 1, which is useful for comparing trees of different sizes.

The tree edit distance from Zhang and Shasha works on ordered trees, which, for our purposes, presents a challenge, as our attack trees are assumed to be unordered. Zhang et al. showed that unordered tree edit distance is a MAX SNP-Hard problem [157]. Our assumption that attack trees do not grow to be very large allows us to remain unconcerned about the processing time of unordered tree distance.

We note that previous work on unordered tree edit distance has offered various polynomial-time approximate solutions, all making assumptions to speed up computations. Fundamentally, all make the same assumption: that there is absolute equivalence between nodes within two trees. Either nodes have identical labels, or they do not.

Just Accepted

This would preclude their use on unfiltered data, as nodes are not guaranteed to have the same label, and indeed, the trees we may want to compare can have widely diverse label sets. On unfiltered data, these algorithms may find a local maximum instead of the global maximum needed to create a mapping (as is done for the label distance described in Section 4.5). As such, these algorithms are unsuitable for our purposes if applied directly.

Applying TED to the examples provided in Figure 4, we follow the process described in [158], by calculating key roots (the rightmost parent of each subtree) and then computing the forest distance. For the specific mechanics of this distance, we refer the reader to the work by Zhang and Shasha [158]. The resulting distance, given a value of 0.7 or less for the semantic similarity threshold ϵ, is 5 (normalized to 0.833). This result demonstrates that TED is unable to handle nodes that are reordered; TED treats these nodes as completely modified.

5.2 Radical Distance (RD)

In our previous work on attack trees, we proposed a mechanism of attack tree decomposition that focused on the concept of *radicals*, or subtrees consisting of a single parent, refinement, and a set of children [125]. Indeed, attack trees are often developed sequentially, by iteratively refining one leaf node at a time [8]. We thus take this decomposition approach and define the radical distance as a way to compare the resulting set of radicals. This approach of comparing substructures has been done before, and relies on the assumption that mapping substructures can represent the overall structure [64, 131].

In Radical Distance (RD), we first decompose an attack tree into a collection of subtrees that are each a unit height, indexed by each subtree's root node. We then perform a semantic comparison of each of the subtree roots; this semantic mapping is the same greedy mapping we conduct in label distance. We calculate the distance subtree by subtree, adding a distance of 1 if the root nodes are not equal, and a distance of 0.5 if the refinements are not equal. For the children, we perform a greedy matching operation similar to label distance. We do not include the distances of children that are themselves roots of other radicals, as these children will be evaluated when roots are compared in another iteration. This process repeats until all children are marked as compared or added/removed.

Applying RD to the examples provided in Figure 4, we first compute the radical decomposition of both attack trees. These are the one height subtrees rooted at *Gain Access* and *Phishing* for Figure 4a and *Gain Unauthorized Access* and *Phishing* for Figure 4b. We then compute the semantic distance between the root nodes of each of the radicals, in a manner similar to LD, taking the most closely related radicals first. In this case, the two radical roots that are identical (*Phishing*) are selected first, and the radical distance is calculated by comparing the root nodes, refinements, and all children (excluding children that themselves are roots of other radicals). In this case, this comparison yields a difference of 0, given a value of 0.7 or less for the semantic similarity threshold ϵ. This is repeated for the other two radicals, which have a distance of 1 (*Brute force*), resulting in an overall distance of 1 (normalized to 0.167).

5.3 Multiset Distance

Attack trees can be represented mathematically in many ways; one well-established attack tree semantics is the multiset semantics proposed by Mauw and Oostdijk [89]. In the multiset semantics, each multiset represents a single complete attack vector within the attack tree, and a set of these multisets represents the whole attack tree. To define the multiset distance, we can use the Jaccard distance between two multisets.

To expand the Jaccard distance to count some similar elements of multisets as identical, we need to calculate the semantic similarity between the elements of the multisets. This would result in a similar problem to the label distance problem or the modification of Zhang and Shasha's tree edit distance, as we do not know a priori which labels should be matched with which ones. Because we assume that attack trees are reasonably small, we perform this calculation by calculating the Jaccard distance between each possible pair of sets, using semantic similarity

Just Accepted

to determine the equivalence of the elements in each set. This is functionally performing label distance on each pair of multisets. We then consider the two sets that have the smallest distance to be identical (a greedy mapping). We then remove these sets from the list of sets and repeat this process until all sets are compared. Ultimately, this results in a calculation that is functionally a nested Jaccard distance.

Multiset semantics, like many mathematical attack tree semantics [57, 89], contain only the leaf node labels. All other labels are reduced from the tree. Additionally, the exact structure of the tree is not generally reconstructible. Ultimately, semantics are meant to answer whether two trees are equivalent, which would mean that the exact structure of the tree is not necessarily important. In attack tree distance, however, the exact structure may be significant. By comparing the differences between multisets, we lose some information about the structure of the tree. We also lose the information contained in any intermediate nodes within the tree.

Applying MSD to the examples provided in Figure 4, we first calculate the multiset semantic representations of both trees. These would be {{`Send Email`, `Receive Credentials`,}, {`Find unlocked computer`}} for Figure 4a and {{`Send Phishing Email`, `Get Credentials`}, {`Find open computer`}, {`Brute force`}} for Figure 4b. We then calculate the Jaccard distance between each possible pair of sets, using semantic similarity to determine the equivalence of the elements in each set. Starting from the pair of sets with the lowest Jaccard distance (the sets {`Find unlocked computer`} and {`Find open computer`}), we add the Jaccard distance of these sets (in this case 0) to the overall distance measure. These sets are then removed from the list of sets. This is repeated until all sets are compared, and any sets remaining (in this case {`Brute force`}) are compared to an empty set. In this case, this yields a distance of 1 (normalized to be 0.167), given a value of 0.7 or less for the semantic similarity threshold ϵ.

5.4 Weighted Sum Distance (WSD)

The four distance measures that we considered previously (LD, TED, RD, and MSD) stem from some intuitions about how attack trees could be compared based on their components. We also consider it interesting to explore whether a combination of these measures would yield interesting results. We thus introduce the weighted sum distance (WSD) as a weighted average of the four baseline distance approaches discussed above.

In the next section, we will first assess the four distance measures described in this section according to our experimental design described in Section 6. We will then search for a combination distance measure that is the weighted sum of the other four distance measures. This weighted sum can be expressed as follows:

$$\alpha_1 * \text{LD} + \alpha_2 * \text{TED} + \alpha_3 * \text{RD} + \alpha_4 * \text{MSD}$$

For a given vector $\alpha = [\alpha_1, \alpha_2, \alpha_3, \alpha_4]$. We will look for an optimal alpha such that the performance of this weighted sum outperforms any individual distance measure shown above.

Applying WSD to the examples provided in Figure 4, we would first calculate LD, TED, RD, and MSD for the two trees. We would then calculate the weighted sum of these values according to the selected value of α to find the WSD. The value of WSD will depend both on the value of ϵ (as it will affect the four primary distance measures) as well as the selected values of α, which affect the sum.

6 Validation Method

In order to evaluate any approach for comparing attack trees, we must first establish a means of validating the results and assessing the quality of this approach. We propose two components for validating the distance measures.

6.1 Experimental Validation

We offer an experimental design that can be used to validate a distance measure on "real-world", human-produced data. This requires a validation dataset that contains two sets of attack tree models. First, we require a base set of

Just Accepted

trees that are identical. This can be created by asking participants to create an attack tree describing the same precisely defined scenario. Any variation between the trees would likewise be representative of "real-world" label variation, and not due to differences in attack steps. In this way, we can evaluate how the distance measures handle similar but not fully identical data.

Second, we require the base set of trees to be slightly expanded by the authors. This results in a second set of trees that are expected to be slightly different, but not very diverse. At the same time, the base set contains attack trees that are subtrees of this second set. This allows us to evaluate the behavior of the distance measures when comparing a tree to a known subset of that tree, and to compare a set of attack trees that are slightly different from each other. This behavior should be predictable, and we can assess distance measures if they display the behavior we expect. We describe the details of our study design and the data collection process below.

6.1.1 Experimental Study Design. We conducted an experiment with students who were assigned a project related to attack trees as a part of their regular coursework. The project was a mandatory, graded assignment. Students had the option to allow for their anonymized responses to be collected for research purposes (opt-in for the study). Our ethical considerations are described in more detail in Section 6.1.3.

The assignment consisted of several components that required students to create an attack tree using a web application we provided. The component of the study that provided data for validating our approach contained two parts. In Part A, students were tasked with creating an attack tree from a provided written scenario; students were instructed to only and fully adapt the written scenario.

The scenario text was created by reading the attack tree in Naik et al. [101] top-down (abstraction level by abstraction level), always going from left to right. Our intention with Part A was for participants to create trees that all reflect the same scenario in roughly the same manner. However, given that participants could elect to phrase nodes differently or include varying amounts of information within nodes, we expected that trees would be similar without being identical. Using this set of trees, we can evaluate how well the distance measures handle "unfiltered" node labels. We further refer to models created in this part as **AT1**.

We then asked students in Part B to expand upon their attack tree by "doing your own research, add at least 5 new nodes and 2 new refinements to the attack tree you created in the previous section". In all, this creates a relatively predictable yet varied dataset. In Part A, all trees should roughly be the same, bearing minor changes in the node labels or the order of nodes. We expect any large differences in Part A to be due to a misunderstanding of the assignment or misreading the scenario. As Part B is built from each student's Part A, we expect that the "core" of their attack tree to remain unchanged; the added nodes and refinement should introduce slightly larger trees that would still be somewhat similar to the original ones. We will refer to the extended attack trees produced in this part as **AT2**.

Finally, we asked students to create a completely new attack tree based on a scenario of their own choosing. These trees should be completely unique and are used in Section 7.6 to validate how the distance measures we defined map similar trees close to each other. The set of completely unique trees is referred to as **AT3**.

Based on previous experience with similar studies, we expect most students to add exactly 5 new nodes and 2 new refinements, which should allow for some predictable distance variations (e.g., the cost of adding 5 new nodes in the tree edit distance). This allows us to evaluate the distance measures as a similarity metric. As each student produced three trees, a base (AT1) and an extension of the base (AT2), and a completely unique tree (AT3), by comparing them, we can validate the distance measures. If the distance measures are not able to recognize the base tree, the distance measure may not be suitable.

Further, we use multi-dimensional scaling (MDS) to map the distance of the trees to a two-dimensional space. This allows us to visualize the distance between the trees and perform manual checks to see if the trees that are mapped close to each other are actually similar.

Just Accepted

6.1.2 Participants. 39 students participated in this study conducted in the fall semester of 2023. They were all third-year bachelor students taking part in a minor on cybersecurity at Leiden University (NL). Most students were in policy-related majors such as Security Studies or International Relations. The participants, on average, had only a few months of programming experience. Students were asked if they had any prior knowledge of threat models or attack trees, and only a few students had any prior knowledge.

6.1.3 Ethics. This study was approved by the Science Faculty Ethics Review Board at Leiden University. Students were informed of the study objectives and design, and gave written consent for their responses to be included in the study. The assignment provided to students was a mandatory, graded, course component, but participation in the study was *opt-in* and fully voluntary. We took significant care to ensure students did not feel pressured to participate. Only (anonymized) data from the students who signed the informed consent form was used in this study. The ethical considerations of this study are very similar to those of our previous study [127]. For more details on the ethical considerations of this study, we thus refer to the ethics considerations section of Schiele and Gadyatskaya [127].

6.2 Theoretical Validation

This validation method involves defining a series of *basic transformation examples* (BTEs) that represent the most fundamental ways in which two attack trees could differ from each other. The idea is that these BTEs only express the transformations that we would expect to see, or that we explicitly would desire to compare two distance measures along. This design has been used previously, with select examples used to model specific behavior of edit distances [3, 139]. In these works, select BTEs are used to model specific behavior of the proposed distance measures and to validate the distance measures.

We define a comprehensive set of BTEs to show all possible basic transformations. For each of these BTEs, we state our intuitive understanding of the distance, and then provide the distance according to our five distance measures. For example, if an order of AND or OR-refined children is changed, this does not affect the underlying attack scenario representation (as both of these refinement operators do not mandate any specific order for executing the attack actions). Thus, the distance between two such order-perturbed attack trees is intuitively expected to be 0. We then provide this BTE to the distance measures and compare it to our intuitive understanding.

We describe and show all of our BTEs in Figure 5. In all of our BTEs except for Figure 5b, we expect the distance between Figure 5a and the provided BTE to be one. In Figure 5b, we expect the distance to be zero.

7 Results

We now discuss the evaluation results for the considered distances that we experimentally evaluated following the design presented in Section 6. The code we use to perform experiments and test our results can be found at the following GitHub repository: https://github.com/nschiele/ATD-TOPS-Data.

We first evaluate the performance of the semantic similarity and the considered distance measures on the human-produced attack tree dataset (the experimental set-up and the dataset description are provided in Section 6.1). Then, by examining the performance of each of our distance measures on our set of BTEs (see Section 6.2), we can evaluate the theoretical validity of each distance measure. Further, we can use the distance according to the BTEs to determine an optimal α for WSD.

7.1 Evaluating alternatives to semantic similarity

As established in Section 4.2, we use SBERT with cosine similarity to establish the similarity between two node labels. This is in direct contrast to a more simplistic approach of using the normalized Levenshtein distance to compare two labels. We have examined the potential effects of using normalized Levenshtein distance (with and without tokenization) on our select approach of SBERT; we present this examination here. In Table 1, we show

Just Accepted

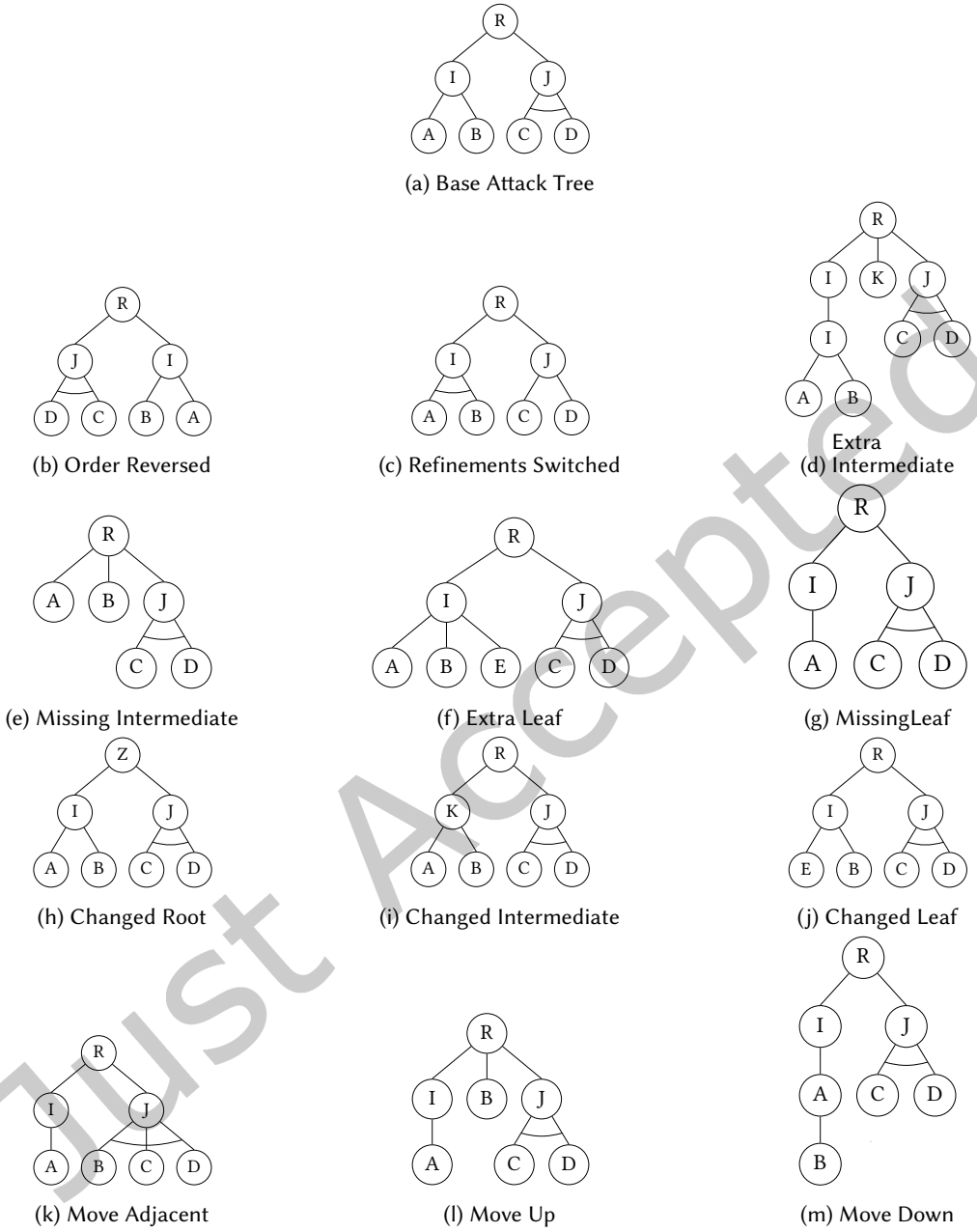


Fig. 5. Basic Transformation Examples (BTEs)

Just Accepted

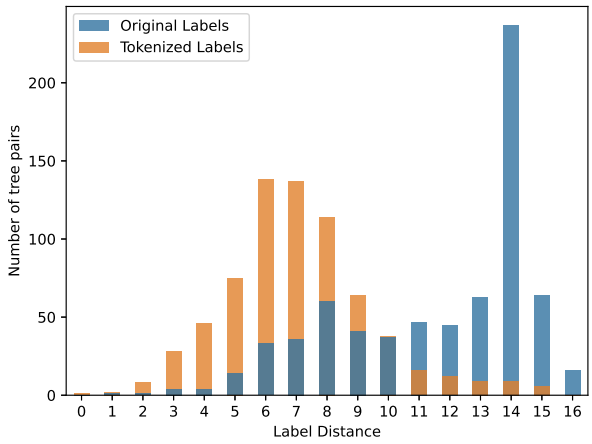


Fig. 6. Pair-wise comparisons of submitted ATs for the same textual scenario (731 total tree pairs) using the normalized Levenshtein distance to determine difference, with and without simple tokenization (lowercase, remove spaces). The collection of these trees is explained in detail in Section 6.1

.

the results of using the normalized Levenshtein distance and SBERT to compare two labels. We can see that the two methods produce very different results, with the normalized Levenshtein distance producing a value of 0 for "door open" and "open door", while the SBERT cosine similarity produces a value of 0.992. This is a clear indication that the two labels are semantically similar, while the normalized Levenshtein distance does not capture this.

A potential mechanism to improve the normalized Levenshtein distance would be to tokenize the strings. Tokenization could include shifting all letters to lowercase, removing spaces, removing prepositions, taking verb stems (removing conjugation), etc. [1, 20, 39, 95]. Although this would certainly reduce the Levenshtein distance, as the selected form of tokenization would remove small differences, this does require decisions concerning which form of tokenization is appropriate and representative for the sample [51]. Further, it will still be the case that the tokenized Levenshtein distance does not accurately reflect the distances between two labels as a human would assess this distance [102].

We see in Figure 6 the pairwise distances of each pair of tree label sets from the AT1 set (where all tree label sets should be the same). Without tokenization, there are no two tree label sets evaluated to be identical; whereas with tokenization, four tree label sets (two distinct pairs) are evaluated to be identical. When a representative sample of label sets was independently evaluated by the authors of this work, we assessed the greatest distance between two label sets to be 4, while the greatest distance between tokenized label sets using Levenshtein distance was 15. This demonstrates that tokenization can be used to improve the accuracy of the Levenshtein distance, but it may not be the best method for comparing labels.

Tokenization cannot solve the underlying issue with using normalized Levenshtein distance: comparison of the labels as strings does not consider the meaning of the labels. Strings with many similar characters may have a small Levenshtein distance while meaning completely disparate things. We demonstrate this in Table 1. By contrast, by using SBERT to compare semantic meaning, we are able to compare the meaning of the labels, rather than the labels themselves. Thus, we used pre-trained models from Sentence-BERT [119] (built on BERT [27]), which create vectors to represent the semantic meaning of the input sentences.

Just Accepted

Label 1	Label 2	Normalized Levenshtein Similarity	SBERT [119] Semantic Similarity
obtain personal data	obtain personnel	0.5882	0.4507
obtain personal data	gather private info	0.1176	0.5521
break open safe	break open door	0.6923	0.7855
break open safe	crack safe open	0.1538	0.814
crack safe	crack door	0.5556	0.6692
door open	open door	0.0	0.992

Table 1. Select examples of potential node label comparisons using both Levenshtein distance and semantic similarity. Semantic similarity is calculated using the methodology shown in Figure 2.

Just Accepted

7.2 Determining Optimal ϵ

In Figure 7, we show our process for establishing optimal ϵ for our dataset.

7.2.1 Sentence Embeddings. As we discussed in Section 4, SBERT is a state-of-the-art method for creating sentence embeddings [119]. We use the `SentenceTransformers` library to calculate the sentence embeddings and cosine similarity to calculate the embedding vector difference. As stated previously, our contribution is not the creation of these embeddings; additionally, our use of sentence embeddings is method agnostic, and any method for generating sentence or word embeddings could be used with our methodology. We used multiple different pre-trained SBERT models and aggregated the results to illustrate that any results we found were not the result of the use of a specific model. Models were selected to be representative of different model sizes, to ensure that the results were not found from using a specifically sized model. Additionally, we did not fine-tune any model. We used the following models: `paraphrase-multilingual-MiniLM-L12`, `all-mpnet-base`, and `all-MiniLM-L12`.

7.2.2 Optimal Threshold. To determine the optimal semantic similarity threshold, ϵ, we follow the process shown in Figure 7. Based on the assignment design, the set of AT1 trees should be identical. We first manually compare a subset of label set pairs, determining if the pairs are indeed equivalent to a human. For some pairs, we made equivalence assumptions, which are documented in Table 2. This is repeated until a sufficient number of pairs of label sets is manually evaluated such that complete sets of equivalent ATs can be created via the transitive property. We then calculate the LD using a decreasing range of ϵ values until there is no distance between any of the known equivalent sets of AT1s. All other node labels either cannot be reasonably assumed to be equivalent or would in all cases be considered equivalent. Possible optimal ϵ values for a subset of possible equivalence assumptions are given in Table 3.

We have elected to accept all the "synonymic assumptions" (A6-8) as true, as we believe that these are simply due to word choices from our participants. We have elected to reject all the "length assumptions" (A1-5) as we believe that participants were including various amounts of information in their labels when labels were significantly different in length, and this indicates that the scenario was being interpreted somewhat differently. This results in an optimal ϵ of 0.7, which is the value we show in the remaining figures as a dotted line.

7.3 Effect of Semantic Similarity

One of our contributions is the application and examination of using semantic similarity to compare node labels when comparing two trees, which would enable the use of tree distance on real-world examples, and not only on curated datasets. Of interest is how our datasets change with rising a semantic similarity limit (ϵ). We expect that for each distance measure, the distance between trees will increase with a rising ϵ; an increasing number of label pairs have a semantic distance above ϵ, nodes that were previously marked as matching (as the cosine

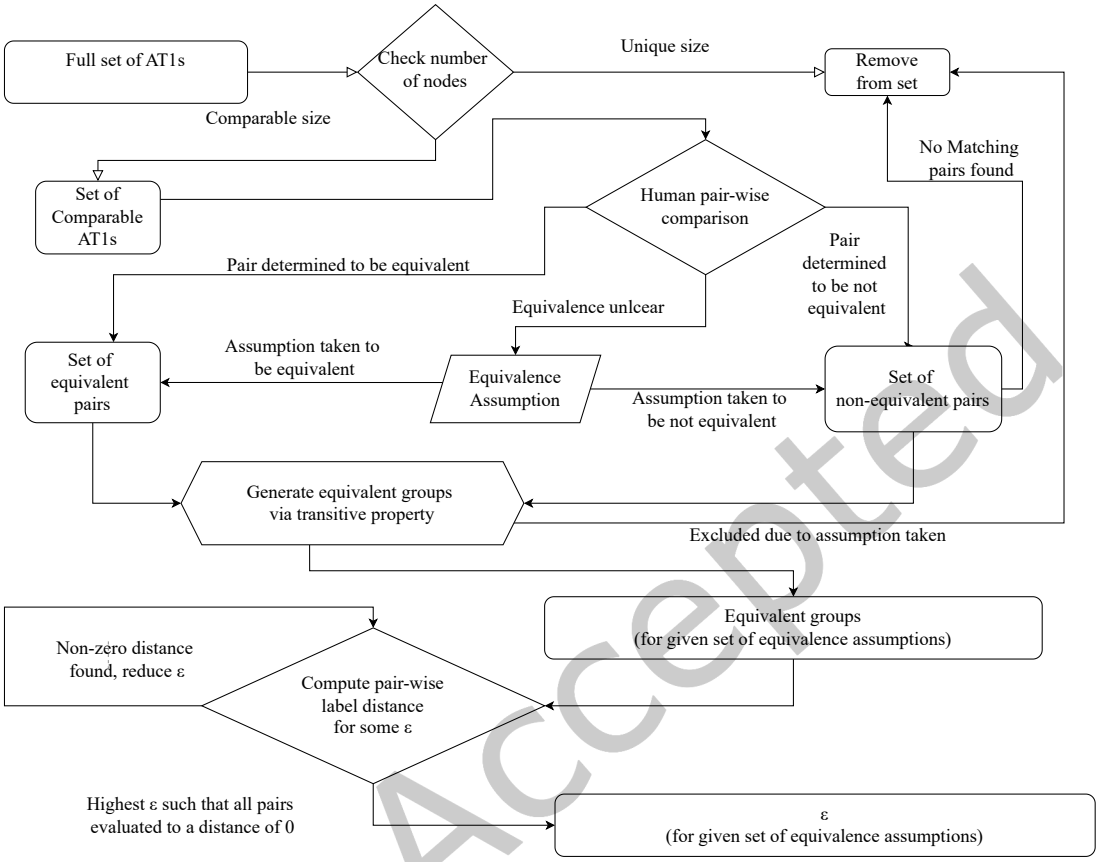


Fig. 7. Flowchart showing the process described in Section 7.2

Just Accepted

similarity of their semantic embeddings was below ϵ) become considered as needing a "change" operation. In Figures 8a and 8b, this is precisely the behavior we see. As described in Section 7.2.1, we use three pre-trained embedding models to ensure that our results are not biased to a particular model. Each solid line is the mean value across the three models of the given distance, with the deviations above and below the mean value represented by the shaded areas.

In Figure 8a, we can see the average normalized distance values for each of the distance measures plotted against an increasing semantic similarity threshold ϵ. As described in Section 6.1.1, AT1 was created by participants creating an attack tree from a written description (the attack tree shown in Figure 1), which should result in identical (or nearly identical) trees. We expect to see distance measures that do not change with rising ϵ, which for distances values below our experimentally determined optimal ϵ value of 0.7 should not change, which is largely what we see to be the case. Further, we can see that for TED and RD, the distances are largely vertically translated from LD, which is a result we would expect; the structural distance measures (TED, RD, and MSD) are built on LD with added syntactic information, and thus should have an increased normalized distance. Of

	Label 1	Label 2	*Average change in* Largest Group	Average Group Size	Total Comparisons	Best ϵ
A1	submit a request for additional data other than what is needed for verification	submit a request for additional data	5.812	1.220	54.859	-0.029
A2	link the dataset via a request for verification	Linking the dataset	1.938	1.305	34.234	-0.044
A3	Unauthorized Access to Profile	Unauthorized Access	0.000	0.289	3.234	-0.010
A4	gaining User credentials and accessing the Profile	Gain user credentials	0.938	0.235	10.859	-0.006
A5	Acquire Unauthorized Access to Profile	Acquire Unauthorized Access	0.000	-0.590	2.016	-0.000
A6	background data attack	get background data	0.938	0.235	10.234	-0.001
A7	steal personal data	obtain personal data	0.438	0.495	8.484	-0.068
A8	personal data extorsion	Steal personal data	0.125	-0.124	2.688	-0.001

Table 2. The eight equivalence assumptions we collected as a result of following the process shown in Figure 7. For each assumption, example labels are provided, along with *the average change in* the number of trees in the largest equivalence group, the average equivalence group size, the total number of pair-wise comparisons, and the best ϵ. These average changes are calculated by taking the difference between all cases where the assumption is taken and not taken, all else being equal. We classified assumptions A1-5 as "length assumptions" where, in general, we need to assume that two labels with varying amounts of information (long & short) are equivalent. We classified assumptions A6-8 as "synonymic assumptions" where we need to assume that two labels with different word choices are synonymous.

A1	A2	A3	A4	A5	A6	A7	A8	Largest Group	Average Group Size	Total Comparisons	Best ϵ
F	F	F	F	F	F	F	F	7	3.5	30	0.779
T	F	F	F	F	F	F	F	11	4.5	66	0.652
F	T	F	F	F	F	F	F	7	4.75	54	0.654
F	F	T	F	F	F	F	F	7	3.75	33	0.645
F	F	F	T	F	F	F	F	7	3.75	31	0.715
F	F	F	F	T	F	F	F	7	3.2	32	0.756
F	F	F	F	F	T	F	F	8	3.75	36	0.735
F	F	F	F	F	F	T	F	7	3.5	30	0.779
F	F	F	F	F	F	F	T	8	3.75	37	0.700
T	T	T	T	T	T	T	T	18	6.75	167	0.573
F	F	F	F	F	T	T	T	9	3.6	46	0.700

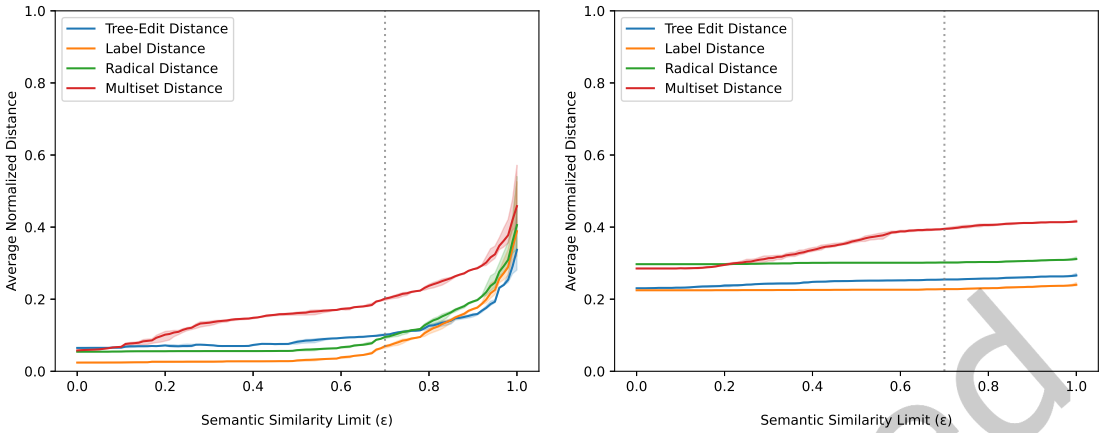
Table 3. A comparison of select different equivalence assumption selections and the effect on the size of the largest equivalence group, the average group size, the total number of pair-wise comparisons, and the best ϵ (taken to be the largest ϵ such that the pairwise distance is 0).

note, MSD does not match the other distance measures, which would appear to indicate that MSD is measuring something other than what is expected.

In Figure 8b, we compare each AT1 to the AT2 generated by the same participant. We would expect all lines to be flat for all distance measures, as the only difference between AT1 and AT2 should be additional nodes, which would be unaffected by ϵ. This is the exact behavior we see for LD, TED, and RD. However, for MSD, we see a different behavior of rising distance with increasing ϵ. This is likely caused by multiset semantics being generated from the leaf nodes only, which means that additional nodes (and especially additional leaf nodes) fundamentally change the multiset semantics representation of a tree. This behavior suggests that MSD may be less suitable as an AT distance measure.

7.3.1 Comparison of Operations. One means of comparison across the different distance measures is in how they differ by operation. In TED, this is the sequence of steps taken to modify one tree into another. In the other distance measures, we can simulate these operations to create a comparable set. Unlike TED, these operations will not necessarily yield a perfectly equivalent tree after modification; however, by counting these "operations" per different similarity limit (ϵ), we can better compare how the different distance calculations arrive at their

Just Accepted



(a) All pairs of AT1 trees

(b) Participant-wise comparison of AT1 to AT2

Fig. 8. Average normalized tree distance for each distance measure for semantic similarity limit (ϵ) ranging from 0 to 1 with steps of 0.01.

final distance calculation. This is a means of comparison between the different distance measures that is meant to provide more insight into *how* the distance measures arrive at a distance value. The difference between these, as affected by the semantic similarity limit ϵ, can provide insight into the behavior and validity of the distance measures.

Figure 9 compares the distribution of operations for the different distance measures. We can see that for LD, RD, and TED, the distribution of operations is relatively consistent across the different similarity limits (ϵ), which is the behavior we would expect (as TED and RD are built on LD). The behavior should normalize when averaged out across the 38 different attack trees, resulting in operations plots that are similar. However, for MSD, we see a different behavior. For MSD, the distribution of operations differs fairly significantly, which is further evidence of the lack of validity of MSD as an AT distance measure.

7.4 Theoretical Evaluation with Basic Transformation Examples (BTEs)

With theoretical validation, we are not examining the ability of distance measures to process "unfiltered" nodes (as this is addressed by the experimental validation). Here, we label all nodes with simple capital letters and set the similarity limit ϵ to 1 (effectively requiring strict equivalence to determine if nodes are equivalent).

As mentioned in Section 6.2, we have 12 examples of various basic transformation examples (BTEs) that could occur between attack trees. These are meant to assess if each of our distance measures is able to capture these changes in a manner we would consider intuitive. The absolute distance for each distance measure for each example is provided in Table 4. There, we describe an "intuitive answer". This is what we believe a practitioner would describe as the distance between the given BTEs and the base attack tree, shown in Figure 5a.

We can see that TED fails to match our intuitive answer on four BTEs, all of these with reordered nodes, as it is not capable of processing nodes in a different order. Further, TED does not match with each of the movement BTEs, as it assigns two operations (removal and addition), which results in a cost of 2. This may be correctable with a modification of the algorithm for checking if the same node is added and removed; however, this would be out of the scope of our work.

Just Accepted

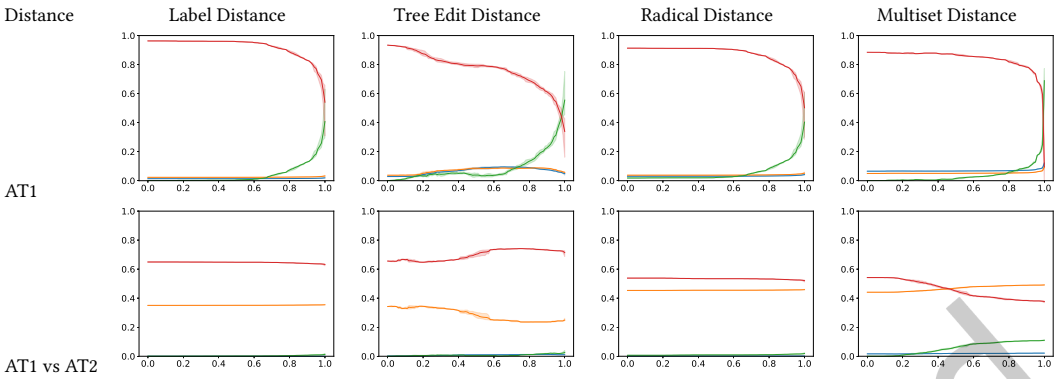


Fig. 9. Plots showing each type of modification operation as a percentage of the total number of operations for increasing similarity limit ϵ. **Blue** indicates a removal operation. **Orange** indicates an addition operation. **Green** indicates a changing operation, and **Red** indicates a matching operation.

Basic Transformation Example (BTE)	Expected Distance	Label Distance	Tree Edit Distance	Radical Distance	Multiset Distance	Weighted Sum Distance α_1	α_2	α_3
Order Reversed	0	0	7	0	0	0	3.5	1.05
Refinement Switch	1	0	1	1	3	0.75	0.5	0.5
Extra Intermediate	1	1	1	1	0	0.85	1	1
Missing Intermediate	1	1	1	4	0	1.75	1	2.05
Extra Leaf	1	1	1	1	1	1	1	1
Missing Leaf	1	1	1	1	1	1	1	1
Changed Root	1	1	1	1	0	.85	1	1
Changed Intermediate	1	1	1	1	0	0.85	1	1
Changed Leaf	1	1	1	1	1	1	1	1
Move Adjacent	1	0	2	2	3	1.05	1	1
Move Up	1	0	2	2	0	0.6	1	1
Move Down	1	0	2	3	1	1.05	1	1.35

Table 4. The distances provided by each of the distance measurements. The expected distance for each of these BTEs is provided in column 1

α_1: We use a weighted value of $\alpha = [0.55, 0, 0.30, 0.15]$

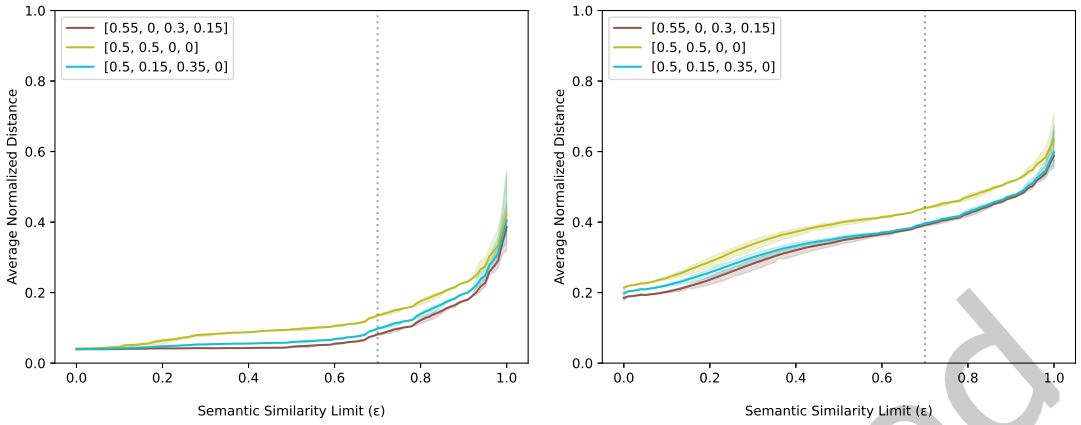
α_2: We use a weighted value of $\alpha = [0.5, 0.5, 0, 0]$

α_3: We use a weighted value of $\alpha = [0.5, 0.15, 0.35, 0]$

Just Accepted

Label distance behaves exactly as expected; it perfectly matches the intuitive answer for all examples in which a label is changed, added, or removed. It does not match the intuition for any example in which the labels all remain the same, but the structure is changed in some way. LD has no mechanism to compare syntax and does not consider the structure of the tree; so long as the tree label sets do not change, neither does LD.

Radical distance nearly matches the intuitive answer except for a few specific situations. In the example of a missing intermediate node, radical distance considers this as needing to remove three nodes (the leaf nodes in the base attack tree and the intermediate) and then re-adding the two leaf nodes. This is caused by the fact that the radicals are mapped between the two trees by their roots, which means that the radical distance has no mechanism to recognize that the two child nodes of one radical best suit another radical. Further, RD faces



(a) WSD on AT1

(b) WSD on AT2

Fig. 10. WSD on AT1 and AT2 using the optimal α value established in Section 7.5

the same issue as TED with moved nodes counting for a larger distance than we expect. As with TED, this is potentially correctable with an algorithm modification or a post-processing step, but this is out of the scope of our work.

Finally, MSD behaves in ways that are not at all intuitive. First, because of the construction of multisets, only the labels of leaf nodes are considered in the calculation. Some structure of the original tree exists in how the multisets are constructed, but reconstruction of the original tree is not possible [89]. Any modification that only applies to intermediate nodes or the root node will not be represented by multiset representations of attack trees, which likewise results in a distance of 0. For changes that affect leaf nodes, multiset semantics seem to over-represent the distance. In the 12 BTEs we created, MSD only matched the intuitive distance in 4 cases. This is further evidence that MSD is not a valid measure of distance between attack trees.

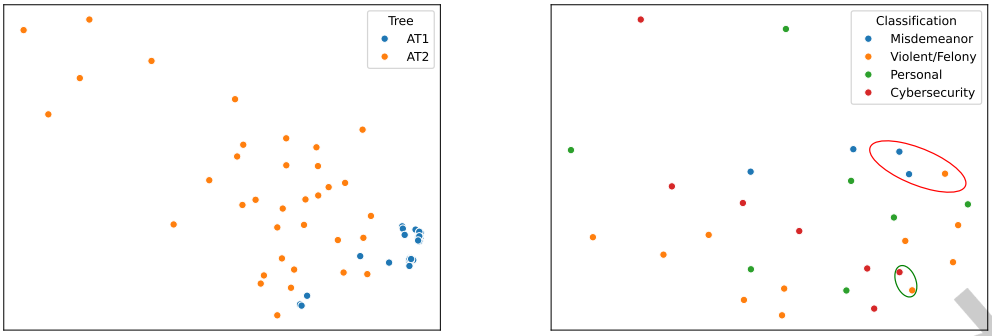
7.5 Determining Optimal α for Weighted Sum Distance

Looking at Table 4, we find that LD, TED, and RD all fail to match our intuitive understanding of distance on four BTEs. However, they all fail on different BTEs. MSD fails to match our intuition on seven of our BTEs.

Using our intuitive understanding of the distance values for our BTEs, we can develop three potential α values for WSD. The first, α_1, is aimed at reducing the difference between our intuitive understanding of distance for each BTE and the distance calculated by WSD. We find this value to be $\alpha_1 = [0.55, 0, 0.3, 0.15]$, and it results in a total deviation of 1.95 from our intuition spread across a total of eight incorrect BTEs. The second, α_2, is aimed at reducing the total number of BTEs that are incorrect. We find this value to be $\alpha_2 = [0.5, 0.5, 0, 0]$, and it results in a total deviation of 4 from our intuition spread across a total of two incorrect BTEs. Finally, we calculate α_3, which is intended to be an optimal value between these two extremes, optimizing for both total incorrect BTEs and total deviation from our intuition. We find this value to be $\alpha_3 = [0.5, 0.15, 0.35, 0]$, and it results in a total deviation of 2.95 from our intuition spread across a total of four incorrect BTEs.

Our three optimal α values are shown in Table 4. Additionally, these three values are shown on AT1 and AT2 with varying ϵ in Figures 10a and 10b. We can see that the variations between the different optimal values of α are relatively minute. Further, we can see the variations caused by the different models for each WSD line are smaller than for the other distances, shown in Figures 8a and 8b; this suggests that WSD may be an ideal way to

Just Accepted



(a) MDS on AT1 and AT2

(b) MDS on AT3

Fig. 11. Multi-dimensional scaling applied to multiple sets of attack trees

Just Accepted

reduce other sources of variation. Overall, we see consistent behavior with WSD for well-chosen α that is in line with our expectations; most importantly, for Figure 10b, the values are consistent across the different distance measures, indicating a reliable distance measure.

7.6 Multi-Dimensional Scaling

Another mechanism to experimentally validate our results is by applying Multi-Dimensional Scaling (MDS). MDS is a technique used to reduce the dimensionality of a dataset [142], in this case, the pair-wise distances between attack trees. We can then visually inspect the resulting 2-dimensional plot and manually compare trees that are plotted close to each other. If we find that closely plotted trees are indeed similar upon manual comparison, this is further evidence that our distance measures are valid.

Given a set of N trees, a pair-wise distance between each tree can be used to generate an $N \times N$ matrix where the i, j value of that matrix will represent the given distance between the ith and jth tree. We can then use multi-dimensional scaling to reduce this $N \times N$ matrix to an $N \times 2$ matrix, which can be plotted on a Cartesian plane [142]. This two-dimensional plot is then more understandable and can subsequently be used to aid analysis. In Figure 11a, we can see MDS applied to both AT1 and AT2 together. We can clearly see the similarity of the AT1 trees (which should all be the same by the experiment design, but are slightly different due to the noise introduced by the participants). For AT2, we can see that they differ from AT1, which is to be expected, but they also vary from each other. If we examine close pairs of AT2s, we can see that the closer they are in the 2-dimensional graph, the more similar the trees appear to be when compared manually.

MDS becomes more powerful when comparing completely different trees, as in the case of AT3. These trees are unique as the scenarios represented were chosen by the participants. In Figure 11b, we can see the 2-dimensional representation of the distances between AT3 trees. Further, we have manually broadly classified the AT3s based on their overall topics, for comparison purposes. We highlight two examples in this plot. The green oval highlights attack trees 27 and 35, which have the root nodes of *Home Burglary* and *Obtain Client Data*, respectively. As we can see, these two trees were manually classified differently; however, upon closer inspection, we find that AT3-35 has many intermediate and leaf nodes that are based around gaining physical access to computer systems, and thus is very similar to AT3-27. The red oval contains trees 5, 18, and 25 with the root nodes *Gain Access to Festival*, *Gain Access to Rugby Game*, and *Steal Mona Lisa*, respectively. The similarity between AT3-5 and AT3-18 should be expected, as both concern gaining access to a physical location. AT3-25 is, judging based on the root

node, very different but still contains many nodes that are similar to AT3-5 and AT3-18, as this scenario considers gaining access to a physical location.

This is a clear example of how MDS can be used to aid in the analysis of attack trees, as an analyst would likely not be able to find this similarity without careful examination of the entire set of trees: a process that would likely take significantly longer and be more prone to error than applying MDS as shown here. Additionally, as the nodes we checked in Figure 11b are similar, this supports the idea that the distance measures are valid, as they are able to capture this similarity, and this is translated on a 2-dimensional graph.

The distances used for MDS in Figures 11 are calculated using the α_3 value shown in Table 4. The distances were normalized. The coordinates of the MDS graph are not provided, as the individual coordinates of a point are not meaningful; only the relative distance between points is meaningful. This is a known quality of MDS [17]. The MDS graph is generated using the scikit-learn implementation of MDS [112].

8 Theoretical Analysis

One important aspect that has yet to be explored is whether the discussed distance measures are *metrics*. This means the following axioms must be satisfied:

$$d(a,a) = 0 \qquad \text{(distance to itself)} \qquad \text{(A1)}$$

$$d(a,b) > 0 \iff a \neq b \qquad \text{(positivity)} \qquad \text{(A2)}$$

$$d(a,b) = d(b,a) \qquad \text{(symmetry)} \qquad \text{(A3)}$$

$$d(a,c) \leq d(a,b) + d(b,c) \qquad \text{(triangle inequality)} \qquad \text{(A4)}$$

for all attack trees a, b, c.

That is, the distance of each attack tree to itself and only to itself should be zero; the distance between two trees should always be positive and symmetric, and for each three attack trees, the triangle inequality must be satisfied by their pair-wise distances [40]. Analyzing which axioms our distance measures fulfill gives us valuable insight into the associated properties of the distance measures and helps us evaluate the application of these distance measures to new data (attack trees) [37]. Further, this analysis helps place our proposed distances in context with a wider range of different distance evaluation measures and metrics [28]. We show below that our distance measures do not fulfill all axioms of metrics and are therefore not metrics. However, our distance measures do consistently fulfill two axioms (A2 and A3), which enable the measures to be compared with similar measures [96] along these axioms, and the conditions under which our metrics violate axioms A1 and A4 provide valuable insight into both how these measures evaluate distance and the limitations of these distance measures [113]. Further, the distance measures we propose can be used to further develop new (potentially metric) measures [94]; finally, in some applications, specifically in matching applications, as we demonstrate, non-metric measures are valuable for improved performance [124].

In this section, we analyze the proposed distances and formally investigate whether they satisfy the metric axioms.

Assumption. Notably, the definition of a metric requires a definition of a unique object. In our use case, following the recursive attack tree definition in Section 2, we can argue that two attack trees are identical if they have the same structure and labels that can be considered identical by a human user (i.e., their distance should be 0). This assumption is operationalized in our label semantic similarity method, where we set up a threshold that defines which labels are identical (similar enough) or different (too dissimilar). This thresholding method can clearly violate the triangle inequality (A4): given text labels a, b and c, such that a and b and b and c are pair-wise similar enough ($>\epsilon$) and thus the distance between them is 0, while the semantic similarity a and c is less than ϵ and thus their distance is greater than 0.

Just Accepted

Label Distance. LD compares two label sets using a greedy algorithm; as such, a comparison of two identical label sets would yield a distance of 0 (A1). However, as LD only compares the label sets of a tree and not the structure of a tree, it may be the case that two differently structured attack trees with the same label set would have a distance of 0; thus, LD only fulfills axiom A2 for label sets and not for the original attack trees: there might be non-identically structured attack trees with LD of 0. Note that, as LD is additive, it fulfills the non-negative requirement of axiom A2. Further, LD trivially fulfills axiom A3. LD is implemented in a manner similar to calculating the Jaccard distance on two sets, and the non-normalized Jaccard distance follows the triangle inequality [73]. However, as LD has the thresholding mechanism described in Section 4.4, we do not implement an identical process to the Jaccard distance. As such, we are unable to confirm if LD fulfills the triangle inequality; further, we cannot confirm if LD is indeed a metric.

Tree Edit Distance. Li and Chenguang proved that the non-normalized TED is a metric [79]. TED trivially fulfills axioms A1 and A3. TED is additive, which means it fulfills the non-negative requirement of axiom A2. Further, as all elements of an attack tree are included in TED (all node labels and structural elements), it is not possible for non-identical trees to have a distance of 0, thus fulfilling the non-zero requirement as well. As such, TED fulfills axiom A2, as we would expect from [79]. However, our implementation of TED uses the LD distance mechanism, and the thresholding mechanism in LD can violate the A4 triangle inequality axiom; thus, our implementation of TED does not fulfill all the axioms of a metric.

Radical Distance. RD only results in a distance of 0 if the radical roots, refinement, and all child nodes are equal (A1). Like TED, RD is additive; as such, it will always fulfill the non-negative requirement of axiom A2. However, as RD only compares trees as a series of single refinement radicals, it is possible for RD between two not fully identical trees to result in a distance of zero, for example, in the event of a tree with repeated nodes or nodes that are similar enough (under the semantic similarity threshold ϵ). As such, RD does not meet the non-zero distance requirement of axiom A2 between non-identical trees. Further investigation is required to prove if this is the only possibility. We leave it for future work, as this demands also a careful investigation of what a unique attack tree is, and providing a definition that practitioners would universally accept. Next, RD trivially fulfills the symmetry axiom (A3). However, as RD is implemented with LD, and LD does not hold axiom A4, it follows that RD does not hold A4, and is therefore not a metric.

Multiset Distance. The multiset distance is similar to LD and the Jaccard distance [73], but only using the leaf nodes of the comparison trees [89]. As with LD, because elements of the trees are not represented (in the case of MSD, the labels of the intermediate and root nodes), it is possible for two trees that have the same structure and leaf nodes but different intermediate node labels to have an MSD of zero, despite these trees not being equivalent trees, thereby violating A2.

The symmetry axiom (A3) is trivial for this distance measure. As with the other distances, MSD does not necessarily hold the triangle inequality (A4) due to the use of the LD distance.

Weighted Sum Distance. As WSD is the weighted sum of other distances, it will be the case that WSD will maintain any of the metric axioms that are held by all other distances included in the sum [40]. Axioms A1 and A3 are held by all distances, and therefore hold for WSD. Axiom A2 is held by LD for label sets but not for attack trees themselves. Further, axiom A2 is not guaranteed to be held by MSD. Therefore, A2 is not maintained for WSD. Finally, as none of the measures satisfy the triangle inequality (A4), this axiom is not maintained for WSD.

9 Discussion

9.1 Label Comparison

Following work in the field of Natural Language Sentence Matching (NLSM) [85, 149], we created a process to extract the labels from pairs of trees and compare them to calculate *label distance (LD)*; a Jaccard distance-based measure between the label sets of two ATs. This distance uses semantic similarity to determine if labels are

Just Accepted

equivalent, using a well-established method of BERT to determine semantic similarity between two labels [27, 143]. This method is able to capture the label similarity between two attack trees. As this method does not require labels to be identical or normalized, it is directly applicable to unfiltered data. We have shown how we can compare human-created sets of labels (**SRQ1**).

9.2 Structural Comparison

We defined three distance measures that are built on LD and consider the refinement structure of ATs alongside the semantic similarity comparison from LD: *attack tree edit distance (TED)*, *radical distance (RD)*, and *multiset distance (MSD)*. TED is based on the Zhang and Shasha tree edit distance algorithm [158], which is a well-established method for comparing trees. RD is a novel distance measure that considers the refinement structure of ATs, based on the refinement structure from [35] and the AT decomposition from [125]. MSD is a distance measure that considers the multiset semantics of ATs from [89].

Finally, we proposed the *weighted sum distance (WSD)*, which is a weighted sum of the previous four distance measures. This is a simple mechanism to combine the distance measures, potentially in such a way that the strengths of each distance measure are combined. With these distance measures, we have shown how we can compare the refinement structure of attack trees (**SRQ2**).

9.3 Validation of Distance Measures

We proposed two methods for validating our distance measures. With theoretical validation, we follow from other tree distance works [92, 111, 158] and propose a set of BTEs to demonstrate the behavior of our distance measures. As with previous works, our BTEs demonstrate the differing behaviors of our distance measures, showing that all distance measures behave as expected in some cases and fail to match some of our other expectations of BTEs. This adds to our suggestion that WSD could be a way to overcome the limitations of individual distance measures, which we attempt to do by suggesting different α values for WSD based on the BTEs. Overall, the BTEs are a method for showing the theoretical validity of distance measures, and we show that LD, RD, TED, and WSD are all theoretically valid in most cases, while MSD is not.

With experimental validation, we presented a repeatable experiment for creating human-generated validation data of two sets of ATs: AT1s, which should be entirely identical, and AT2s, which should be mostly identical to the AT created by the same participant. We then used this data to both find an optimal semantic similarity limit ϵ (in a thoroughly described and repeatable manner), as well as verify the distance measures we have found. We also used the freely designed trees (AT3s) to examine if the distance measures place similar trees close together, and a qualitative assessment of this mapping shows that it does, validating our distance measures. We find that LD, RD, TED, and WSD are all experimentally valid, while MSD is not. Further, by using MDS to plot WSD on a 2-dimensional plane, and qualitatively assessing trees plotted close together, we can further verify that these distance measures actually represent the difference and similarity of attack trees. We thus have shown how we can validate our distance measures for attack trees (**SRQ3**).

These results directly support the practical applications discussed in Section 1, which motivate this work. The MDS clustering in Section 7.6 demonstrates how the measures reveal similarities between attack trees, while the consistent separation of AT1 and AT2 in Section 7.3 shows how they can support categorization in crowd-sourced AT libraries [29] and validation of LLM-synthesized trees against human references [36].

9.4 Limitations

We now discuss the key limitations of this work.

Underlying modules. SBERT is a popular yet not the most advanced model to establish the semantic similarity of sentences [32, 49, 69]. Our intention was to develop a method that can be easily iterated upon, replicated, and

Just Accepted

improved. Using SBERT as a means of establishing semantic similarity meets these goals, as it is a well-established and easily accessible tool. Still, it could be replaced with a more advanced model in the future. The same reasoning applies to the use of the simple Zhang and Shasha tree edit distance algorithm. While it is not the most advanced tree edit distance algorithm, it is well-established and easily accessible. In the future, it could be replaced with a more advanced or task-specific algorithm.

Semantic similarity limitations. The distance measurements we evaluate with these data should be applicable to any attack trees created. However, it may be the case that the optimal ϵ and α may be different for attack trees from different data sets or those made by experts. We have extensively documented our methodology for finding these values and believe that this exact process could be repeated for any new dataset.

Limitations of our experimental validation. As our validation method relies on human-produced attack trees, it is an open question whether these studied attack trees in sets AT1, AT2, and AT3 are representative enough of real-life, practitioner-designed attack trees. Previous work by Naiakshina et al. and Karpati et al. suggests that in the context of cybersecurity, students may be a sufficient proxy for practitioners [63, 100]. However, students possess limited knowledge of security concepts and thus a limited vocabulary to design sufficiently diverse and interesting attack trees. In the future, studies can try to capture ATs designed by experts following the same experimental procedure.

Limitations of the proposed distance measures. As our evaluation has shown, the distance measures we define are not yet a perfect solution for determining the similarity of two attack trees. We propose a first step toward this goal, and we believe that our work paves the way for future research in this area. The initial tree edit distance by Tai [139] was not the final word on tree distance measures; instead it was the first step in a long line of research that has led to the development of many tree distance measures, with no clear definite methodology yet established [92, 107, 111, 158, 159]; this is described in detail in Section 3. We thus believe that our work will be the first step in a similar line of research for attack trees.

10 Conclusion and Future Work

We proposed several new approaches for defining the distance between attack trees and offered a comprehensive comparison between these measures. We have designed and demonstrated a robust and repeatable validation scheme for these measures. To the best of our knowledge, our work is the first to examine the problem of attack tree similarity. In the future, we plan to examine other approaches to combine the studied measures to assess the quality of automatically generated attack trees. Further, we could apply more advanced sentence matching techniques or investigate other tree distances from other fields to benchmark and improve these measures. Following from our analysis in Section 8, we plan to investigate exactly under which circumstances the triangle inequality is held, and to conduct further experimentation to exactly define what experts consider to be "equivalent" trees. We could also explore the possibility of new semantic representations of attack trees that would enable direct comparison or enhance our distance measures. Finally, these techniques can be expanded to be applied to other graphical threat models, including, notably, attack graphs, by taking advantage of the transformation approach defined in the literature [115, 125].

Just Accepted

Acknowledgments

We thank the anonymous reviewers for their insightful feedback and suggestions that helped us to improve this work. An early version of this paper was plagiarized and published by a third party. We are grateful for the support we received from LIACS and Leiden University in having the plagiarized paper retracted.

References

[1] L. A. Mullen, K. Benoit, O. Keyes, D. Selivanov, and J. Arnold. 2018. Fast, Consistent Tokenization of Natural Language Text. *Journal of Open Source Software* 3, 23 (2018), 655. doi:10.21105/joss.00655

[2] M. Adams. 2024. STRIDEGPT. Online. https://github.com/mrwadams/stride-gpt.
[3] T. Akutsu, T. Mori, N. Nakamura, S. Kozawa, Y. Ueno, and T. N. Sato. 2021. Tree Edit Distance with Variables. Measuring the Similarity between Mathematical Formulas. arXiv:2105.04802 [cs]
[4] M. Albanese, S. Jajodia, and S. Noel. 2012. Time-efficient and cost-effective network hardening using attack graphs. In *IEEE/IFIP International Conference on Dependable Systems and Networks (DSN 2012)*. IEEE, IEEE, Boston, MA, USA, 1–12.
[5] P. Ammann, D. Wijesekera, and S. Kaushik. 2002. Scalable, graph-based network vulnerability analysis. In *Proceedings of the 9th ACM Conference on Computer and Communications Security*. ACM, Washington, DC, USA, 217–224.
[6] R. Anderson. 2020. *Security Engineering: A Guide to Building Dependable Distributed Systems*. John Wiley & Sons, West Sussex, England.
[7] S. Aseervatham, E. Viennet, and Y. Bennani. 2007. A Semantic Kernel for Semi-structured DocumentS. In *Seventh IEEE International Conference on Data Mining (ICDM 2007)*. IEEE, New York, NY, 403–408. doi:10.1109/ICDM.2007.23 ISSN: 2374-8486.
[8] M. Audinot, S. Pinchinat, and B. Kordy. 2018. Guided design of attack trees: a system-based approach. In *2018 IEEE 31st Computer Security Foundations Symposium (CSF)*. IEEE, Oxford, 61–75.
[9] K. Beckers, M. Heisel, L. Krautsevich, F. Martinelli, R. Meis, and A. Yautsiukhin. 2014. Determining the probability of smart grid attacks by combining attack tree and attack graph analysis. In *Smart Grid Security: Second International Workshop, SmartGridSec 2014, Munich, Germany, February 26, 2014, Revised Selected Papers 2*. Springer, Munich, Germany, 30–47.
[10] J. Bellemans, M. Raciti, D. Van Landuyt, L. Sion, G. Bella, L. Desmet, and W. Joosen. 2025. From Cluttered to Clustered: Addressing Privacy Threat Explosion through Automated Clustering. In *2025 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*. IEEE, Venice, Italy, 231–241.
[11] S. Bengio, J. Weston, and D. Grangier. 2010. Label Embedding Trees for Large Multi-Class Tasks. In *Advances in Neural Information Processing Systems*, Vol. 23. Curran Associates, Inc., New York, NY, 1–10.
[12] K. Bernsmed, D. S. Cruzes, M. G. Jaatun, and M. Iovan. 2022. Adopting threat modelling in agile software development projects. *Journal of Systems and Software* 183 (2022), 111090. doi:10.1016/j.jss.2021.111090
[13] G. S. Bopche and B. M. Mehtre. 2017. Graph similarity metrics for assessing temporal changes in attack surface of dynamic networks. *Computers & Security* 64 (Jan. 2017), 16–43.
[14] J. Bryans, L. S. Liew, H. N. Nguyen, G. Sabaliauskaite, S. Shaikh, and F. Zhou. 2020. A template-based method for the generation of attack trees. In *Proc. of WISTP*. Springer, New York, NY, 155–165.
[15] J. Bryans, L. S. Liew, H. N. Nguyen, G. Sabaliauskaite, and S. A. Shaikh. 2023. Formal Template-Based Generation of Attack–Defence Trees for Automated Security Analysis. *Information* 14, 9 (2023), 481.
[16] E. Cambria and B. White. 2014. Jumping NLP Curves: A Review of Natural Language Processing Research [Review Article]. *IEEE Computational Intelligence Magazine* 9, 2 (05 2014), 48–57. doi:10.1109/MCI.2014.2307227 Conference Name: IEEE Computational Intelligence Magazine.
[17] J. D. Carroll and P. Arabie. 1998. Multidimensional scaling. *Measurement, judgment and decision making* 31, 607 - 49 (1998), 179–250.
[18] D. Chandrasekaran and V. Mago. 2022. Evolution of Semantic Similarity—A Survey. *Comput. Surveys* 54, 2 (2022), 1–37. doi:10.1145/3440755
[19] S. Chen and K. Zhang. 2014. An improved algorithm for tree edit distance with applications for RNA secondary structure comparison. *Journal of Combinatorial Optimization* 27, 4 (05 2014), 778–797.
[20] S. Choo and W. Kim. 2023. A study on the evaluation of tokenizer performance in natural language processing. *Applied Artificial Intelligence* 37, 1 (12 2023), 2175112. doi:10.1080/08839514.2023.2175112 Publisher: Taylor & Francis _eprint: https://doi.org/10.1080/08839514.2023.2175112.
[21] K. Chowdhary. 2020. Natural language processing. *Fundamentals of artificial intelligence* 1, 51-89 (2020), 603–649.
[22] M. Collins and N. Duffy. 2002. Convolution Kernels for Natural Language. In *Advances in Neural Information Processing Systems 14*, Thomas G. Dietterich, Suzanna Becker, and Zoubin Ghahramani (Eds.). The MIT Press, Cambridge, MA, 625–632. doi:10.7551/mitpress/1120.003.0085
[23] R. Collobert, J. Weston, L. Bottou, M. Karlen, K. Kavukcuoglu, and P. Kuksa. 2011. Natural Language Processing (Almost) from Scratch. *Journal of Machine Learning Research* 12 (2011), 2493–2537.
[24] M. Cuculovic, F. Fondement, M. Devanne, J. Weber, and M. Hassenforder. 2022. Semantics to the rescue of document-based XML diff: A JATS case study. *Software: Practice and Experience* 52, 6 (2022), 1496–1516.
[25] A. De Nicola, A. Formica, I. Mele, and F. Taglino. 2025. Information Content Methods for Semantic Similarity: An Experimental Assessment. *IEEE Access* PP (2025), 1–1. doi:10.1109/ACCESS.2025.3584192
[26] A. De Nicola, A. Formica, M. Missikoff, E. Pourabbas, and F. Taglino. 2023. A parametric similarity method: Comparative experiments based on semantically annotated large datasets. *Journal of Web Semantics* 76 (2023), 1–10. Issue C. doi:10.1016/j.websem.2023.100773
[27] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *North American Chapter of the Association for Computational Linguistics*. ACL, New York, NY, 4172–4186.
[28] M. M. Deza and E. Deza. 2014. Generalizations of Metric Spaces. In *Encyclopedia of Distances*. Springer, New York, NY, 71–84.

Just Accepted

[29] F. Dorfhuber, J. Eisentraut, K. Klioba, and Jan Křetínskỳ. 2024. QuADTool: Attack-Defense-Tree Synthesis, Analysis and Bridge to Verification. In *International Conference on Quantitative Evaluation of Systems and Formal Modeling and Analysis of Timed Systems*. Springer, Munich, Germany, 52–71.
[30] A. Duncan, S. Creese, and M. Goldsmith. 2019. A combined attack-tree and kill-chain approach to designing attack-detection strategies for malicious insiders in cloud computing. In *2019 International Conference on Cyber Security and Protection of Digital Services (Cyber Security)*. IEEE, New York, NY, 1–9.
[31] I. Elsharef, Z. Zeng, and Z. Gu. 2024. Facilitating Threat Modeling by Leveraging Large Language Models. In *Workshop on AI Systems with Confidential Computing*. NDSS, San Diego, CA, 1–8.
[32] J. Fodor, S. D. Deyne, and S. Suzuki. 2025. Compositionality and Sentence Meaning: Comparing Semantic Parsing and Transformers on a Challenging Sentence Similarity Dataset. *Computational Linguistics* 51, 1 (March 2025), 139–190. doi:10.1162/coli_a_00536
[33] M. Fraile, M. Ford, O. Gadyatskaya, R. Kumar, M. Stoelinga, and R. Trujillo-Rasua. 2016. Using attack-defense trees to analyze threats and countermeasures in an ATM: a case study. In *The Practice of Enterprise Modeling: 9th IFIP WG 8.1. Working Conference, PoEM 2016, Skövde, Sweden, November 8-10, 2016, Proceedings 9*. Springer, Skövde, Sweden, 326–334.
[34] E. Gabrilovich and S. Markovitch. 2007. Computing semantic relatedness using Wikipedia-based explicit semantic analysis.. In *IJCAI'07: Proceedings of the 20th international joint conference on Artifical intelligence*, Vol. 7. ACM, Hyderabad, India, 1606–1611.
[35] O. Gadyatskaya, R. Jhawar, S. Mauw, R. Trujillo-Rasua, and T. A. C. Willemse. 2017. Refinement-Aware Generation of Attack Trees. In *Security and Trust Management (Lecture Notes in Computer Science)*. Springer, Cham, 164–179.
[36] O. Gadyatskaya and D. Papuc. 2023. ChatGPT Knows Your Attacks: Synthesizing Attack Trees Using LLMs. In *International Conference on Data Science and Artificial Intelligence*. Springer, Leiden, Netherlands, 245–260.
[37] T. M. Ghazal. 2021. Performances of k-means clustering algorithm with different distance metrics. *Intelligent Automation & Soft Computing* 30, 2 (2021), 735–742.
[38] V. A. Gonzalez-Huitron, A. E. Rodriguez-mata, L. E. Amabilis-Sosa, R. Baray-Arana, I. Robledo-Vega, and G. Valencia-Palomo. 2023. Jaccard distance as similarity measure for disparity map estimation. *IEEE Latin America Transactions* 21, 5 (2023), 690–698.
[39] G. Grefenstette. 1999. Tokenization. In *Syntactic Wordclass Tagging*, Hans van Halteren (Ed.). Springer Netherlands, Dordrecht, 117–133. doi:10.1007/978-94-015-9273-4_9
[40] M. Gromov. 2007. Metric structures for Riemannian and non-Riemannian spaces, english ed., Modern Birkhäuser Classics.
[41] A. Haddad, N. Aaraj, P. Nakov, and S. Mare. 2024. Automated Mapping of CVE Vulnerability Records to MITRE CWE Weaknesses. In *Proceedings of the 21st International Conference on Security and Cryptography (SECRYPT 2024)*. SCITEPRESS – Science and Technology Publications, Dijon, France, 500–507. doi:10.48550/arXiv.2304.11130
[42] W. Han, J. Xue, and K. Qian. 2021. A Novel Malware Detection Approach Based on Behavioral Semantic Analysis and LSTM Model. In *2021 IEEE 21st International Conference on Communication Technology (ICCT)*. IEEE, Tianjin, China, 339–343. doi:10.1109/ICCT52962.2021.9658113 ISSN: 2576-7828.
[43] A. Happe and J. Cito. 2023. Getting pwn'd by ai: Penetration testing with large language models. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, San Francisco, CA, 2082–2086.
[44] Y. He, Y. Wang, S. Zhu, W. Wang, Y. Zhang, Q. Li, and A. Yu. 2024. Automatically Identifying CVE Affected Versions With Patches and Developer Logs. *IEEE Transactions on Dependable and Secure Computing* 21, 2 (2024), 905–919. doi:10.1109/TDSC.2023.3264567 Conference Name: IEEE Transactions on Dependable and Secure Computing.
[45] Z. He, C. Wu, G. Liu, Z. Zheng, and Y. Tian. 2015. Decomposition tree: a spatio-temporal indexing method for movement big data. *Cluster Computing* 18, 4 (2015), 1481–1492. doi:10.1007/s10586-015-0475-3
[46] J. Hirschberg and C. D. Manning. 2015. Advances in natural language processing. *Science* 349, 6245 (2015), 261–266. doi:10.1126/science.aaa8685 Publisher: American Association for the Advancement of Science.
[47] J. Homer, A. Varikuti, X. Ou, and M. A. McQueen. 2008. Improving Attack Graph Visualization through Data Reduction and Attack Grouping. In *Visualization for Computer Security*. Vol. 5210. Springer, Cambridge, MA, USA, 68–79.
[48] B. Hu, Z. Lu, H. Li, and Q. Chen. 2014. Convolutional Neural Network Architectures for Matching Natural Language Sentences. In *Advances in Neural Information Processing Systems*, Vol. 27. Curran Associates, Inc., Montreal, Canada, 2042–2050.
[49] Y. Huang, S. Zhu, W. Liu, J. Wang, and X. Wei. 2025. Addressing Asymmetry in Contrastive Learning: LLM-Driven Sentence Embeddings with Ranking and Label Smoothing. *Symmetry* 17, 5 (May 2025), 646.
[50] S. Hussain, A. Kamal, S. Ahmad, and G. Rasool. 2014. THREAT MODELLING METHODOLOGIES: A SURVEY. *Sci.Int.(Lahore* 26 (01 2014), 1607–1609.
[51] 2010. *Handbook of Natural Language Processing* (2 ed.). Chapman and Hall/CRC, New York. doi:10.1201/9781420085938
[52] T. R. Ingoldsby. 2010. Attack tree-based threat risk analysis. *Amenaza Technologies Limited* 1 (2010), 3–9.
[53] M. G. Ivanova, C. W. Probst, R. Hansen, and F. Kammüller. 2015. Attack tree generation by policy invalidation. In *Proc. of WISTP (LNCS, Vol. 9311)*. Springer, Copenhagen, Denmark, 249–259.

Just Accepted

[54] S. Jajodia, S. Noel, and B. O'berry. 2005. Topological analysis of network attack vulnerability. In *Managing Cyber Threats: Issues, Approaches, and Challenges*. Springer, Boston, MA, USA, 247–266.
[55] A. Jaquith. 2007. *Security metrics*. Pearson Education, London, England.
[56] S. Jha, O. Sheyner, and J. Wing. 2002. Two formal analyses of attack graphs. In *Proceedings 15th IEEE Computer Security Foundations Workshop. CSFW-15*. IEEE, Verona, Italy, 49–63. doi:10.1109/CSFW.2002.1021806
[57] R. Jhawar, B. Kordy, S. Mauw, S. Radomirović, and R. Trujillo-Rasua. 2015. Attack Trees with Sequential Conjunction. In *ICT Systems Security and Privacy Protection (IFIP Advances in Information and Communication Technology)*. Springer, Hamburg, Germany, 339–353.
[58] R. Jhawar, K. Lounis, S. Mauw, and Y. Ramírez-Cruz. 2018. Semi-automatically augmenting attack trees using an annotated attack tree library. In *Proc. of STM'18*, Vol. LNCS 11091. Springer, Barcelona, Spain, 85–101.
[59] J. J. Jiang and D. W. Conrath. 1997. Semantic Similarity Based on Corpus Statistics and Lexical Taxonomy. In *Proceedings of the 10th Research on Computational Linguistics International Conference*, Keh-Jiann Chen, Chu-Ren Huang, and Richard Sproat (Eds.). The Association for Computational Linguistics and Chinese Language Processing (ACLCLP), Taipei, Taiwan, 19–33.
[60] P. Johnson, Robert Lagerström, and Mathias Ekstedt. 2018. A meta language for threat modeling and attack simulations. In *Proceedings of the 13th international conference on availability, reliability and security*. ACM, Hamburg, Germany, 1–8.
[61] N. Kalchbrenner, E. Grefenstette, and P. Blunsom. 2014. A Convolutional Neural Network for Modelling Sentences. In *Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Baltimore, Maryland, 655–665. doi:10.3115/v1/P14-1062
[62] E. Kapetanios, D. Tatar, and C. Sacarea. 2013. *Natural Language Processing: Semantic Aspects*. CRC Press, Boca Raton, Florida.
[63] P. Karpati, Y. Redda, A. L. Opdahl, and G. Sindre. 2014. Comparing attack trees and misuse cases in an industrial setting. *Information and Software Technology* 56, 3 (March 2014), 294–308.
[64] H. Kashima and T. Koyanagi. 2002. Kernels for Semi-Structured Data. In *Proceedings of the Nineteenth International Conference on Machine Learning*. MK, Sydney, Australia, 291–298.
[65] S. Katsikeas, E. Rencelj Ling, P. Johnsson, and M. Ekstedt. 2024. Empirical evaluation of a threat modeling language as a cybersecurity assessment tool. *Computers & Security* 140 (May 2024), 103743.
[66] H. Kaur, C. Powers, R. E. T. Iii, S. Fahl, and D. Votipka. 2025. "Threat modeling is very formal, it's very technical, and also very hard to do correctly": Investigating Threat Modeling Practices in Open-Source Software Projects. In *34th USENIX Security Symposium*. USENIX, Seattle, WA, 2125–2144.
[67] K. Kaynar. 2016. A taxonomy for attack graph generation and usage in network security. *Journal of Information Security and Applications* 29 (Aug. 2016), 27–56.
[68] H. Kekul, B. Ergen, and H. Arslan. 2022. Comparison and Analysis of Software Vulnerability Databases. *International Journal of Engineering and Manufacturing* 12, 4 (2022), 1.
[69] Y. Kim, D. Oh, and H. H. Huang. 2025. SynCSE: Syntax Graph-based Contrastive Learning of Sentence Embeddings. *Expert Systems with Applications* 287 (May 2025), 128047. doi:10.1016/j.eswa.2025.128047
[70] H.-K. Kong, M. K. Hong, and T.-S. Kim. 2018. Security risk assessment framework for smart car using the attack tree analysis. *Journal of Ambient Intelligence and Humanized Computing* 9 (2018), 531–551.
[71] A.-M. Konsta, A. L. Lafuente, B. Spiga, and N. Dragoni. 2024. Survey: Automatic generation of attack trees and attack graphs. *Computers & Security* 137 (2024), 103602.
[72] B. Kordy, S. Mauw, S. Radomirovic, and P. Schweitzer. 2014. Attack-defense trees. *Journal of Logic and Computation* 24, 1 (2014), 55–87.
[73] S. Kosub. 2019. A note on the triangle inequality for the Jaccard distance. *Pattern Recognition Letters* 120 (2019), 36–38.
[74] L. Krautsevich, F. Martinelli, and A. Yautsiukhin. 2010. Formal approach to security metrics.: what does "more secure" mean for you?. In *Proceedings of the Fourth European Conference on Software Architecture: Companion Volume*. ACM, Copenhagen, Denmark, 162–169.
[75] P. Lakkaraju, S. Gauch, and M. Speretta. 2008. Document similarity based on concept tree distance. In *Proceedings of the nineteenth ACM conference on Hypertext and hypermedia*. ACM, Pittsburgh, PA, USA, 127–132.
[76] H. S. Lallie, K. Debattista, and J. Bal. 2017. An Empirical Evaluation of the Effectiveness of Attack Graphs and Fault Trees in Cyber-Attack Perception. *IEEE Trans. on Information Forensics and Security* 13 (2017), 14.
[77] V. Levenshtein. 1966. Binary codes capable of correcting deletions, insertions, and reversals. In *Cybernetics and Control Systems*, Vol. 10. Soviet physics doklady, Moscow, USSR, 707–710.
[78] B. Li and L. Han. 2013. Distance weighted cosine similarity measure for text classification. In *Intelligent Data Engineering and Automated Learning–IDEAL 2013: 14th International Conference, IDEAL*. Springer, Hefei, China, 611–618.
[79] Y. Li and Z. Chenguang. 2011. A metric normalization of tree edit distance. *Frontiers of Computer Science in China* 5 (2011), 119–125.
[80] T. Liu, X. Wang, C. Lv, R. Zhen, and G. Fu. 2020. Sentence Matching with Syntax- and Semantics-Aware BERT. In *Proceedings of the 28th International Conference on Computational Linguistics*. International Committee on Computational Linguistics, Barcelona, Spain (Online), 3302–3312.
[81] Y. Liu, M. Gardner, and M. Lapata. 2018. Structured Alignment Networks for Matching Sentences. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Brussels, Belgium, 1554–1564.

Just Accepted

[82] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov. 2019. RoBERTa: A Robustly Optimized BERT Pretraining Approach. arXiv:1907.11692 [cs]

[83] O. Lo, W. J. Buchanan, P. Griffiths, and R. Macfarlane. 2018. Distance Measurement Methods for Improved Insider Threat Detection. *Security and Communication Networks* 2018, 1 (2018), 5906368.

[84] A. Long. 2024. Attack Tree GPT. Online. https://github.com/aulong-msft/AttackTreeGPT.

[85] X. Lu, Y. Deng, T. Sun, Y. Gao, J. Feng, X. Sun, and R. Sutcliffe. 2022. MKPM: Multi keyword-pair matching for natural language sentences. *Applied Intelligence* 52, 2 (2022), 1878–1892. doi:10.1007/s10489-021-02306-5

[86] P. X. Mai, A. Goknil, L. K. Shar, F. Pastore, L. C. Briand, and S. Shaame. 2018. Modeling security and privacy requirements: a use case-driven approach. *Information and Software Technology* 100 (2018), 165–182.

[87] A. Marzal and E. Vidal. 1993. Computation of normalized edit distance and applications. *IEEE transactions on pattern analysis and machine intelligence* 15, 9 (1993), 926–932.

[88] W. J. Masek and M. S. Paterson. 1980. A faster algorithm computing string edit distances. *J. Comput. System Sci.* 20, 1 (1980), 18–31. doi:10.1016/0022-0000(80)90002-1

[89] S. Mauw and M. Oostdijk. 2006. Foundations of Attack Trees. In *Information Security and Cryptology - ICISC 2005 (Lecture Notes in Computer Science)*, Dong Ho Won and Seungjoo Kim (Eds.). Springer, Berlin, Heidelberg, 186–198. doi:10.1007/11734727_17

[90] S. McLaughlin, D. Podkuiko, and P. McDaniel. 2010. Energy theft in the advanced metering infrastructure. In *Critical Information Infrastructures Security (CRITIS)*. Springer, Athens, Greece, 176–187.

[91] Robert C. McNamee. 2013. Can't see the forest for the leaves: Similarity and distance measures for hierarchical taxonomies with a patent classification example. *Research Policy* 42, 4 (2013), 855–873. doi:10.1016/j.respol.2013.01.006

[92] Matt McVicar, Benjamin Sach, Cédric Mesnage, Jefrey Lijffijt, Eirini Spyropoulou, and Tijl De Bie. 2016. SuMoTED: An intuitive edit distance between rooted unordered uniquely-labelled trees. *Pattern Recognition Letters* 79 (2016), 52–59. doi:10.1016/j.patrec.2016.04.012

[93] L. Meng, J. Gu, and Z. Zhou. 2012. A New Hybrid Semantic Similarity Measure Based on WordNet. In *Network Computing and Information Security*, Jingsheng Lei, Fu Lee Wang, Mo Li, and Yuan Luo (Eds.). Springer, Berlin, Heidelberg, 739–744. doi:10.1007/978-3-642-35211-9_93

[94] M.-L. Michelsohn. 1982. On the existence of special metrics in complex geometry. *ACTA Mathematica* 149 (1982), 261–295.

[95] S. J. Mielke, Z. Alyafeai, E. Salesky, C. Raffel, M. Dey, M. Gallé, A. Raja, C. Si, W. Y. Lee, B. Sagot, and S. Tan. 2021. Between words and characters: A Brief History of Open-Vocabulary Modeling and Tokenization in NLP. *ArXiv* 1 (2021), 1–27.

[96] P. W. Mielke Jr. 1986. Non-metric statistical analyses: some metric alternatives. *Journal of Statistical Planning and Inference* 13 (1986), 377–387.

[97] T. Mikolov, K. Chen, G. Corrado, and J. Dean. 2013. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781* 3781 (2013), 1–12.

[98] A. Moschitti. 2006. Making tree kernels practical for natural language learning. In *ACL*. ACL, New York, NY, 113–120.

[99] L. Muñoz-González, D. Sgandurra, A. Paudice, and E. C. Lupu. 2017. Efficient Attack Graph Analysis through Approximate Inference. *ACM Transactions on Privacy and Security* 20, 3 (Aug. 2017), 1–30.

[100] A. Naiakshina, A. Danilova, E. Gerlitz, and M. Smith. 2020. On Conducting Security Developer Studies with CS Students: Examining a Password-Storage Study with CS Students, Freelancers, and Company Developers. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*. ACM, Honolulu, HI, USA, 1–13.

[101] N. Naik, P. Grace, P. Jenkins, K. Naik, and J. Song. 2022. An evaluation of potential attack surfaces based on attack tree modelling and risk matrix applied to self-sovereign identity. *Computers & Security* 120 (Sept. 2022), 102808. doi:10.1016/j.cose.2022.102808

[102] A. Nayak, H. Timmapathini, K. Ponnalagu, and V. Gopalan Venkoparao. 2020. Domain adaptation challenges of BERT in tokenization and sub-word representations of Out-of-Vocabulary words. In *Proceedings of the First Workshop on Insights from Negative Results in NLP*, Anna Rogers, João Sedoc, and Anna Rumshisky (Eds.). Association for Computational Linguistics, Online, 1–5. doi:10.18653/v1/2020.insights-1.1

[103] S. M. Nicoletti, Milan Lopuhaä-Zwakenberg, Mariëlle Stoelinga, Fabio Massacci, and Carlos E Budde. 2024. How hard can it be? Quantifying MITRE attack campaigns with attack trees and cATM logic. *arXiv preprint arXiv:2410.06692* 1 (2024), 1–31.

[104] S. Noel, M. Elder, S. Jajodia, P. Kalapa, S. O'Hare, and K. Prole. 2009. Advances in topological vulnerability analysis. In *2009 Cybersecurity Applications & Technology Conference for Homeland Security*. IEEE, IEEE, Washington, DC, USA, 124–129.

[105] H. H. Otu and K. Sayood. 2003. A new sequence distance measure for phylogenetic tree construction. *Bioinformatics* 19, 16 (2003), 2122–2130. doi:10.1093/bioinformatics/btg295

[106] X. Ou, S. Govindavajhala, and A. W. Appel. 2005. MulVAL: A Logic-based Network Security Analyzer. In *Proceedings of the 14th USENIX Security Symposium*. USENIX Association, Baltimore, MD, USA, 8–23.

[107] Benjamin Paaßen. 2021. An A*-algorithm for the Unordered Tree Edit Distance with Custom Costs. In *Similarity Search and Applications: 14th International Conference, SISAP 2021, Dortmund, Germany, September 29 – October 1, 2021, Proceedings* (Dortmund, Germany). Springer-Verlag, Berlin, Heidelberg, 364–371. doi:10.1007/978-3-030-89657-7_27

Just Accepted

[108] M. Palmer, D. Gildea, and P. Kingsbury. 2005. The Proposition Bank: An Annotated Corpus of Semantic Roles. *Computational Linguistics* 31, 1 (2005), 71–106. doi:10.1162/0891201053630264
[109] S. Paul. 2014. Towards automating the construction & maintenance of attack trees: a feasibility study. In *Proceedings of GraMSec*. Electronic Proceedings of Theoretical Computer Science, Grenoble, France, 1–16.
[110] S. Paul and R. Vignon-Davillier. 2014. Unifying traditional risk assessment approaches with attack trees. *Journal of Information Security and Applications* 19, 3 (2014), 165–181.
[111] M. Pawlik and N. Augsten. 2011. RTED: A Robust Algorithm for the Tree Edit Distance. arXiv:1201.0230 [cs] doi:10.48550/arXiv.1201.0230
[112] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, and V. Dubourg. 2011. Scikit-learn: Machine learning in Python. *The Journal of Machine Learning Research* 12 (2011), 2825–2830.
[113] Elżbieta Pękalska, Artsiom Harol, Robert PW Duin, Barbara Spillmann, and Horst Bunke. 2006. Non-Euclidean or non-metric measures can be informative. In *Joint IAPR international workshops on statistical techniques in pattern recognition (SPR) and structural and syntactic pattern recognition (SSPR)*. Springer, Heidelberg, Germany, 871–880.
[114] C. Phillips and L. P. Swiler. 1998. A graph-based system for network-vulnerability analysis. In *Proceedings of the 1998 Workshop on New Security Paradigms*. ACM, Charlottesville, VA, USA, 71–79.
[115] S. Pinchinat, M. Acher, and D. Vojtisek. 2015. ATSyRa: an integrated environment for synthesizing attack trees. In *Proc. of GraMSec'15 (LNCS)*. Springer, Verona,Italy, 97–101.
[116] G. Pirró. 2009. A semantic similarity metric combining features and intrinsic information content. *Data & Knowledge Engineering* 68, 11 (2009), 1289–1308. doi:10.1016/j.datak.2009.06.008
[117] A. Rashid, S. A. A. Naqvi, R. Ramdhany, M. Edwards, R. Chitchyan, and M. A. Babar. 2016. Discovering "unknown known" security requirements. In *Proceedings of ICSE*. ACM, Austin,TX, 866–876.
[118] T. Rashid, I. Agrafiotis, and J. R. Nurse. 2016. A New Take on Detecting Insider Threats: Exploring the Use of Hidden Markov Models. In *Proceedings of the 8th ACM CCS International Workshop on Managing Insider Security Threats (MIST '16)*. ACM, Vienna, Austria, 47–56.
[119] N. Reimers and I. Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Hong Kong, 1–11.
[120] D. Ren, S. Du, and H. Zhu. 2011. A novel attack tree based risk assessment approach for address privacy preservation in the VANETs. In *2011 IEEE International Conference on Communications (ICC)*. IEEE, Kyoto, Japan, 1–5.
[121] P. Resnik. 1995. Using information content to evaluate semantic similarity in a taxonomy. In *Proceedings of the 14th international joint conference on Artificial intelligence - Volume 1 (IJCAI'95)*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 448–453.
[122] L. S. Riza, F. S. Anwar, E. F. Rahman, C. U. Abdullah, and S. Nazir. 2020. Natural Language Processing and Levenshtein Distance for Generating Error Identification Typed Questions on TOEFL. *Journal of Computers for Society* 1, 1 (2020), 1–23. doi:10.17509/jcs.v1i1.24940 Number: 1.
[123] L. Santoso, B. Singh, S. Rajest, R. Regin, and K. Kadhim. 2018. A Genetic Programming Approach to Binary Classification Problem. *EAI Endorsed Transactions on Energy Web* 8, 31 (July 2018), 165523. doi:10.4108/eai.13-7-2018.165523
[124] W. J. Scheirer, M. J. Wilber, M. Eckmann, and T. E. Boult. 2014. Good recognition is non-metric. *Pattern Recognition* 47, 8 (2014), 2721–2731. doi:10.1016/j.patcog.2014.02.018
[125] N. D. Schiele and O. Gadyatskaya. 2021. A novel approach for attack tree to attack graph transformation. In *International Conference on Risks and Security of Internet and Systems*. Springer, Ames, IA, 74–90.
[126] N. D. Schiele and O. Gadyatskaya. 2025. Empirical assessment of the perception of graphical threat model acceptability. In *2025 32nd Asia-Pacific Software Engineering Conference (APSEC)*. IEEE, Macau, China, 255–266.
[127] N. D. Schiele and O. Gadyatskaya. 2025. A limited technical background is sufficient for attack-defense tree acceptability. In *Proceedings of USENIX Security Symposium*. USENIX, Seattle, WA, 2225–2244.
[128] B. Schneier. 1999. Attack trees. *Dr. Dobb's journal* 24, 12 (1999), 21–29.
[129] B. Schneier. 2000. *Attacks*. John Wiley & Sons, Ltd, West Sussex, England. 23–41 pages. doi:10.1002/9781119183631
[130] O. Sheyner, J. Haines, S. Jha, R. Lippmann, and J. Wing. 2002. Automated generation and analysis of attack graphs. In *Proceedings 2002 IEEE Symposium on Security and Privacy*. IEEE, Oakland, CA, 273–284. doi:10.1109/SECPRI.2002.1004377 ISSN: 1081-6011.
[131] K. Shin. 2015. A theory of subtree matching and tree kernels based on the edit distance concept. *Annals of Mathematics and Artificial Intelligence* 75, 3 (2015), 419–460. doi:10.1007/s10472-015-9467-5
[132] K. Shin and T. Kuboyama. 2014. A Comprehensive Study of Tree Kernels. In *New Frontiers in Artificial Intelligence*, Yukiko Nakano, Ken Satoh, and Daisuke Bekki (Eds.). Springer International Publishing, Cham, 337–351. doi:10.1007/978-3-319-10061-6_22
[133] A. Shostack. 2014. *Threat modeling: Designing for security*. John Wiley & Sons, West Sussex, England.
[134] M. A. Siddiqi, R. M. Seepers, M. Hamad, V. Prevelakis, and C. Strydis. 2018. Attack-tree-based Threat Modeling of Medical Implants.. In *PROOFS*. Journal of Cryptographic Engineering (JCEN), Amstersdam, The Netherlands, 32–49.
[135] G. Sidorov, F. Velasquez, E. Stamatatos, A. Gelbukh, and L. Chanona-Hernández. 2014. Syntactic N-grams as machine learning features for natural language processing. *Expert Systems with Applications* 41, 3 (2014), 853–860. doi:10.1016/j.eswa.2013.08.015

Just Accepted

[136] B. Solhaug and K. Stølen. 2014. The CORAS Language – why it is designed the way it is. In *Safety, Reliability, Risk and Life-Cycle Performance of Structures and Infrastructures*, George Deodatis, Bruce Ellingwood, and Dan Frangopol (Eds.). CRC Press, New York, NY, 3155–3162. doi:10.1201/b16387-456
[137] K. Sowka, V. Palade, H. Jadidbonab, P. Wooderson, and H. Nguyen. 2023. A review on automatic generation of attack trees and its application to automotive cybersecurity. *Artificial Intelligence and Cyber Security in Industry* 4 (2023), 165–193.
[138] K. J. Sowka, M. Cheah, T. Doan, H. N. Nguyen, and S. Shaikh. 2021. Towards Generation of Attack Trees using Machine Learning. In *15th EuroSys Doctoral Workshop*. ACM, New York, NY, 1–4.
[139] K.-C. Tai. 1979. The Tree-to-Tree Correction Problem. *Journal of the ACM* 26, 3 (1979), 422–433. doi:10.1145/322139.322143
[140] I. Tarandach and M. J. Coles. 2020. *Threat Modeling*. O'Reilly Media, Inc., Sebastopo, CA.
[141] C.-W. Ten, C.-C. Liu, and M. Govindarasu. 2007. Vulnerability assessment of cybersecurity for SCADA systems using attack trees. In *2007 IEEE Power Engineering Society General Meeting*. IEEE, Tampa, FL, 1–8.
[142] W. S. Torgerson. 1952. Multidimensional scaling: I. Theory and method. *Psychometrika* 17, 4 (1952), 401–419.
[143] J. Tracz, P. I. Wójcik, K. Jasinska-Kobus, R. Belluzzo, R. Mroczkowski, and I. Gawlik. 2020. BERT-based similarity learning for product matching. In *Proceedings of Workshop on Natural Language Processing in E-Commerce*, Huasha Zhao, Parikshit Sondhi, Nguyen Bach, Sanjika Hewavitharana, Yifan He, Luo Si, and Heng Ji (Eds.). Association for Computational Linguistics, Barcelona, Spain, 66–75.
[144] J. Turian, L. Ratinov, and Y. Bengio. 2010. Word representations: a simple and general method for semi-supervised learning. In *Proceedings of the 48th Annual Meeting of the Association for Computational Linguistics* (Uppsala, Sweden) *(ACL '10)*. Association for Computational Linguistics, USA, 384–394.
[145] R. Vigo, F. Nielson, and H. R. Nielson. 2014. Automated generation of attack trees. In *Proc. of CSF'14*. IEEE, Vienna, Austria, 337–350.
[146] I. Wagner and D. Eckhoff. 2018. Technical Privacy Metrics: A Systematic Survey. *ACM Comput. Surv.* 51, 3, Article 57 (June 2018), 38 pages. doi:10.1145/3168389
[147] L. Wang, S. Jajodia, A. Singhal, P. Cheng, and S. Noel. 2013. k-zero day safety: A network security metric for measuring the risk of unknown vulnerabilities. *IEEE Transactions on Dependable and Secure Computing* 11, 1 (2013), 30–44.
[148] P. Wang, W.-H. Lin, P.-T. Kuo, H.-T. Lin, and T. C. Wang. 2012. Threat risk analysis for cloud security based on attack-defense trees. In *2012 8th International Conference on Computing Technology and Information Management (NCM and ICNIT)*, Vol. 1. IEEE, Seoul, South Korea, 106–111.
[149] Z. Wang, W. Hamza, and R. Florian. 2017. Bilateral Multi-Perspective Matching for Natural Language Sentences. In *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence*. International Joint Conferences on Artificial Intelligence publisher, Melbourne, Australia, 4144–4150. doi:10.24963/ijcai.2017/579
[150] Wojciech Wideł, Maxime Audinot, Barbara Fila, and Sophie Pinchinat. 2019. Beyond 2014: Formal Methods for Attack Tree–based Security Modeling. *ACM Computing Surveys (CSUR)* 52, 4 (2019), 1–36.
[151] J. Xiao. 2024. A hybrid model to improve IC-related metrics of semantic similarity between words. *Complex & Intelligent Systems* 10, 5 (2024), 6339–6377. doi:10.1007/s40747-024-01496-y
[152] J. Xu, J. W. Stokes, G. McDonald, X. Bai, D. Marshall, S. Wang, A. Swaminathan, and Z. Li. 2024. Autoattacker: A large language model guided system to implement automatic cyber-attacks. *arXiv preprint arXiv:2403.01038* 1 (2024), 1–19.
[153] J.-C. Xu, K. Shin, and Y.-L. Liu. 2016. Detecting Fake Sites based on HTML Structure Analysis. In *Proceedings of the 6th International Conference on Communication and Network Security (ICCNS '16)*. Association for Computing Machinery, New York, NY, USA, 86–90. doi:10.1145/3017971.3017980
[154] T. Yoshino, S. Higuchi, and K. Hirata. 2013. A Dynamic Programming A* Algorithm for Computing Unordered Tree Edit Distance. In *2013 Second IIAI International Conference on Advanced Applied Informatics*. IEEE, Matsue, Japan, 135–140. doi:10.1109/IIAI-AAI.2013.71
[155] L. Yujian and L. Bo. 2007. A Normalized Levenshtein Distance Metric. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 29, 6 (2007), 1091–1095. doi:10.1109/TPAMI.2007.1078
[156] C. Zhang, L. Wang, D. Fan, J. Zhu, T. Zhou, L. Zeng, and Z. Li. 2024. VTT-LLM: Advancing Vulnerability-to-Tactic-and-Technique Mapping through Fine-Tuning of Large Language Model. *Mathematics* 12, 9 (2024), 1286.
[157] K. Zhang and T. Jiang. 1994. Some MAX SNP-hard results concerning unordered labeled trees. *Inform. Process. Lett.* 49, 5 (1994), 249–254. doi:10.1016/0020-0190(94)90062-0
[158] K. Zhang and D. Shasha. 1989. Simple Fast Algorithms for the Editing Distance Between Trees and Related Problems. *SIAM J. Comput.* 18 (Dec. 1989), 1245–1262. doi:10.1137/0218082
[159] K. Zhang, R. Statman, and D. Shasha. 1992. On the editing distance between unordered labeled trees. *Inform. Process. Lett.* 42, 3 (May 1992), 133–139. doi:10.1016/0020-0190(92)90136-J
[160] Y. Zhang, T. Du, Y. Ma, X. Wang, Y. Xie, G. Yang, Y. Lu, and E.-C. Chang. 2025. AttacKG+: Boosting attack graph construction with large language models. *Computers & Security* 150 (2025), 104220.
[161] H. Zhou, W. Huang, M. Li, and Y. Lai. 2022. Relation-Aware Entity Matching Using Sentence-BERT. *EBSCOhost* 71 (2022), 1581–1595.

Received 21 May 2025; revised 22 May 2026; accepted 17 September 2026

Just Accepted